K1Wi Framework

User Manual

Version: v1.2.0 User Manual

Author:

Gregory B. Novak

Release Date:

June 2026

Document Information

Document Title:

K1Wi Framework User Manual

Version:

v1.2.0 User Manual

Document Status:

Final Release

Target Audience:

Cybersecurity Students
Security Researchers
Security Analysts
CTF Participants

Operating System:

Linux

About Document:

This manual provides operational guidance, usage examples, and technical reference material for the K1Wi Framework. It is intended for Cybersecurity students, security researchers, security analysts, and Capture-The-Flag participants seeking a unified toolkit for digital forensics, reverse engineering, binary analysis, and cryptographic investigations.

Table of Contents

Chapter 1 – What Is K1Wi?

1.1 Introduction

K1Wi is an integrated cybersecurity analysis framework that combines reverse engineering, digital forensics, cryptanalysis, binary analysis, and Capture-The-Flag (CTF) tooling into a unified command-line environment.

The goal of K1Wi is to provide security professionals, students, researchers, and enthusiasts with a single toolkit capable of analyzing files, binaries, encoded data, cryptographic artifacts, and forensic evidence without requiring multiple disconnected utilities.

Unlike many specialized tools that focus on only one discipline, K1Wi brings together capabilities from several cybersecurity domains. A user may extract files from an archive, inspect a Linux executable, analyze hidden data within an image, evaluate cryptographic material, calculate Position Independent Executable (PIE) offsets, and perform string intelligence analysis from the same framework.

1.2 Mission

The mission of K1Wi is to provide practical cybersecurity analysis capabilities through a lightweight, transparent, and extensible platform.

K1Wi is designed to help users:

- Analyze suspicious files and artifacts
- Perform reverse engineering tasks
- Conduct digital forensic investigations
- Solve cryptographic and CTF challenges
- Inspect executable binaries
- Identify encoded, hidden, or suspicious data
- Learn cybersecurity concepts through hands-on experimentation

1.3 Target Audience

K1Wi is intended for:

- Cybersecurity students
- Capture-The-Flag competitors
- Security researchers
- Reverse engineers
- Malware analysts
- Digital forensic investigators
- Incident responders
- Hobbyists and self-directed learners

No advanced programming knowledge is required to begin using K1Wi, although users with experience in operating systems, networking, programming, and information security will benefit from the framework's advanced capabilities.

1.4 Design Philosophy

K1Wi is built around several core principles:

1.4.1 Simplicity

Common tasks should require minimal commands and configuration.

1.4.2 Transparency

Analysis results should be understandable and traceable. K1Wi attempts to explain findings rather than simply report them.

1.4.3 Practicality

Features are selected based on real-world usefulness for security practitioners, students, and challenge solvers.

1.4.4 Modularity

Capabilities are organized into independent modules that can evolve without affecting the entire framework.

1.4.5 Education

K1Wi serves as both an operational toolkit and a learning platform for understanding cybersecurity concepts.

1.5 Core Capability Areas

K1Wi currently provides functionality in the following areas:

1.5.1 String Intelligence

Detection and analysis of encoded, encrypted, structured, and suspicious strings.

1.5.2 Digital Forensics

Entropy analysis, file extraction, file carving, embedded signature detection, and artifact discovery.

1.5.3 Image Analysis

JPEG forensic analysis, entropy heat maps, RS steganography detection, Huffman fingerprinting, and DCT coefficient analysis.

1.5.4 Binary Analysis

ELF inspection, symbol enumeration, dependency analysis, and executable metadata extraction.

1.5.5 Reverse Engineering

PIE base calculations, symbol resolution, address translation, and runtime analysis support.

1.5.6 Cryptanalysis

RSA challenge analysis, factorization support, key inspection, and educational cryptographic tooling.

1.5.7 CTF Support

Integrated workflows that assist with challenge analysis, artifact inspection, and investigative problem solving.

1.6 Project Status

Current Release:
Version: v1.2.0 User Manual

Current Release Status:
K1Wi Framework v1.2.0 release documentation

K1Wi continues to evolve through additional analysis modules, expanded testing coverage, improved documentation, and user-driven enhancements.

The remainder of this manual describes installation, operation, workflows, command references, troubleshooting procedures, and development guidance for the K1Wi Framework.

Chapter 2 – Installation and Building K1Wi

2.1 Overview

This chapter describes how to install dependencies, obtain the K1Wi source code, compile the framework, run tests, and verify a successful installation.

K1Wi is developed and tested primarily on Ubuntu Linux and other Debian-based distributions. The build system uses GNU Make and supports standard, debug, release, sanitizer, and thread-sanitizer builds.

2.2 System Requirements

Minimum recommended system:

- Ubuntu 22.04 LTS or newer
- 4 GB RAM
- 500 MB free disk space
- GCC or Clang compiler
- GNU Make

Recommended system:

- Ubuntu 24.04 LTS
- 8 GB RAM or greater
- Clang compiler
- Git
- AddressSanitizer support

2.3 Required Dependencies

K1Wi depends on several external development libraries.

Install all required packages using:

```
sudo apt update

sudo apt install build-essential clang make git libgmp-dev libjpeg-dev libssl-dev libncurses-dev
```

2.4 Dependency Purpose

The following packages provide the compiler, build tools, and development libraries required by K1Wi.

clang
Primary compiler used during development.

make
Build system used to compile K1Wi from source.

git
Source code retrieval and version control.

libgmp-dev
Large integer arithmetic library used by the RSA analysis modules.

libjpeg-dev
JPEG processing library used by image analysis and steganography modules.

libssl-dev
Cryptographic library support used by hashing and cryptographic workflows.

libncurses-dev
Terminal interface library used for text-based user interface support.

2.5 Obtaining the Source Code

Clone the repository:

```
git clone https://github.com/GregNovak/K1Wi.git

cd K1Wi
```

2.6 Directory Structure

A typical K1Wi source tree contains:

```
K1Wi/
├── bin/
├── build/
├── Manuals/
```

```
├── tests/
├── testdata/
├── include/
├── *.c
├── *.h
├── makefile
└── README.md
```

2.6.1 Directory Descriptions

The K1Wi source tree is organized into directories and files that separate compiled output, documentation, tests, sample data, shared code resources, and build instructions.

bin/
: Contains compiled K1Wi executables, including the main k1wi binary after a successful build.

build/
: Stores intermediate object files and temporary build artifacts created during compilation.

Manuals/
: Contains user documentation, release manuals, PDFs, changelogs, roadmap notes, and related documentation files.

tests/
: Contains the automated regression test suite used to validate framework functionality before release.

testdata/
: Contains sample files, RSA challenge inputs, archives, images, binaries, and other artifacts used by tests and examples.

include/
: Contains shared header files used by multiple K1Wi source modules.

***.c**
: C source files that implement K1Wi modules, command handling, utilities, and analysis engines.

***.h**
: Header files that define shared structures, function declarations, constants, and module interfaces.

makefile
: Defines the build process, compiler settings, build modes, and cleanup targets.

README.md
: Provides the main project overview, build instructions, usage notes, and release information.

2.7 Building K1Wi

2.7.1 Clean Build

Remove previous build artifacts:

```
make clean
```

Compile the framework:

```
make
```

Expected output:

```
[CC] ...
[LINK] bin/k1wi
```

Upon successful completion, the executable will be located at:

```
bin/k1wi
```

2.8 Build Modes

2.8.1 Debug Build

Used during development:

```
make debug
```

Debug builds include symbols and additional diagnostic information.

2.8.2 Release Build

Used for production releases:

```
make release
```

Release builds prioritize optimization and reduced binary size.

2.8.3 Sanitizer Build

Used to detect memory safety issues:

```
make sanitize
```

Sanitizer builds help identify:

- Buffer overflows
- Use-after-free bugs
- Invalid memory access
- Memory corruption

2.8.4 Thread Sanitizer Build

Used to detect race conditions in multithreaded code:

```
make tsan
```

2.9 Verifying Installation

After compilation completes, verify that K1Wi starts correctly.

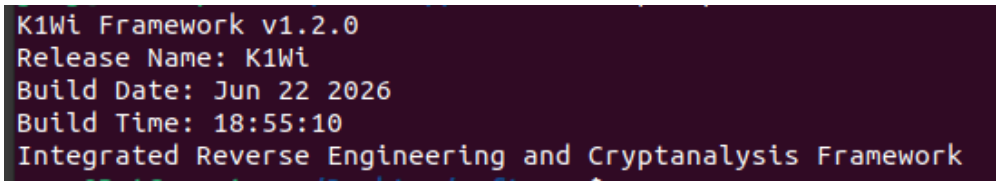
2.9.1 Display Version Information

./bin/k1wi --version

Expected output:

K1Wi Framework v1.2.0
Build Date: ...
Build Time: ...

Figure 2.1 – K1Wi Version Output



Running the version command confirms that the compiled K1Wi binary starts correctly and reports the expected v1.2.0 release information.

2.9.2 Display Help Information

./bin/k1wi HELP

This command displays the available modules and command categories.

The HELP command provides a quick overview of available K1Wi modules, including string analysis, forensic analysis, binary analysis, cryptographic tools, file utilities, and testing workflows.

2.10 Running Regression Tests

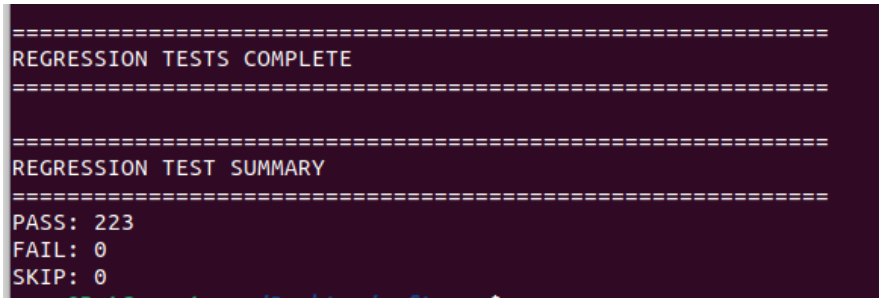
K1Wi includes an automated regression suite.

Execute:

```
./tests/run_regression.sh
```

A successful run will end with output similar to:

Figure 2.2 – Regression Test Summary



The regression test summary confirms that K1Wi's automated validation suite completed successfully before release. Any failures should be investigated before deploying or releasing K1Wi.

2.11 Running Sanitizer Validation

For maximum confidence before release, rebuild K1Wi with AddressSanitizer enabled and run the regression suite again:

make sanitize

./tests/run_regression.sh

AddressSanitizer checks for memory-safety issues such as invalid memory access, buffer overflows, and use-after-free bugs. A successful regression run under AddressSanitizer provides additional confidence that the framework operates correctly under memory-safety instrumentation.

2.12 Troubleshooting

2.12.1 Missing Header Files

Error:

```
fatal error: header.h: No such file or directory
```

Solution:

Verify that all required development packages are installed.

2.12.2 Missing Libraries

Error:

```
undefined reference to ...
```

Solution:

Verify installation of:

- libgmp-dev
- libjpeg-dev
- libssl-dev
- libncurses-dev

2.12.3 Permission Denied

Error:

```
Permission denied
```

Solution:

Verify executable permissions:

```
chmod +x ./bin/k1wi
```

2.12.4 Build Artifacts Corrupted

Solution:

Perform a clean rebuild:

```
make clean
make
```

2.13 Summary

At this point, K1Wi should compile successfully, pass regression testing, and be ready for operational use. The next chapter introduces the Quick Start Guide and demonstrates common workflows that can be performed within the first fifteen minutes of using K1Wi.

Chapter 3 – Quick Start Guide

3.1 Overview

This chapter introduces common K1Wi workflows that new users can run shortly after installation. By the end of this chapter, users should be able to display command help, analyze strings, inspect files, run forensic analysis, examine binaries, and test RSA challenge inputs.

The examples in this chapter are intended as a quick operational tour rather than a complete reference. Later chapters explain each module in greater detail.

3.2 Verify Installation

Display version information:

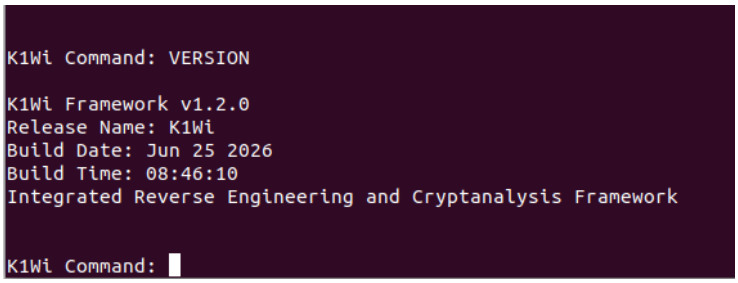
./bin/k1wi --version

Expected output:

K1Wi Framework v1.2.0
Build Date: ...
Build Time: ...
Integrated Reverse Engineering and Cryptanalysis Framework

K1Wi also supports checking version information from the interactive command prompt by entering VERSION.

Figure 3.1 – Interactive K1Wi Version Command



The interactive command prompt allows users to run K1Wi commands from within the framework shell. In this example, the VERSION command displays the active framework version, release name, build date, and build time. If version information appears, the executable is functioning correctly.

3.3 Display Available Commands

Show the built-in command reference:

```
./bin/k1wi HELP
```

This command displays the major K1Wi modules and supported workflows.

Important modules include:

- STRING
- EXTRACT
- LYZER
- ENTROPY
- ELFINFO
- PIECALC
- RSA analysis tools
- Hashing utilities
- File identification utilities

3.4 Analyze a Simple String

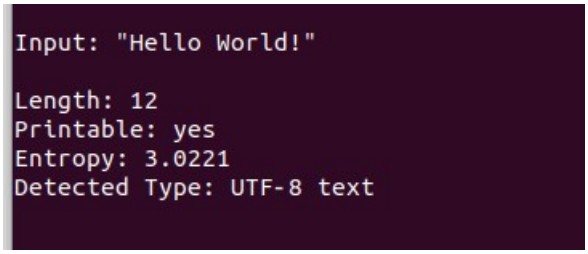
Command:

```
./bin/k1wi STRING "Hello World!"
```

Example output:

```
Length: 12
Printable: yes
Entropy: ...
Detected Type: UTF-8 text
```

Figure 3.2 – Basic STRING Analysis



K1Wi automatically attempts to identify string types and detect common encodings. Simple string analysis is often the fastest way to verify that the command interface is working correctly. The STRING command analyzes text input, reports basic characteristics, and identifies common string types.

3.5 Analyze a Suspicious String

Command:

```
./bin/k1wi STRING "R2d2d2d2d2d424547494e20505249564154452045204b45592d"
```

K1Wi will identify this as a shifted hexadecimal encoded private key marker. This demonstrates the framework's ability to recognize encoded artifacts commonly encountered during investigations and CTF challenges.

3.6 Measure File Entropy

Command:

```
./bin/k1wi ENTROPY <file>
```

Example output:

```
Global entropy: 7.638774 bits/byte
```

High entropy may indicate:

- Compression
- Encryption
- Packed executables
- Embedded payloads

Entropy is an indicator, not proof. Analysts should combine entropy results with other findings before drawing conclusions.

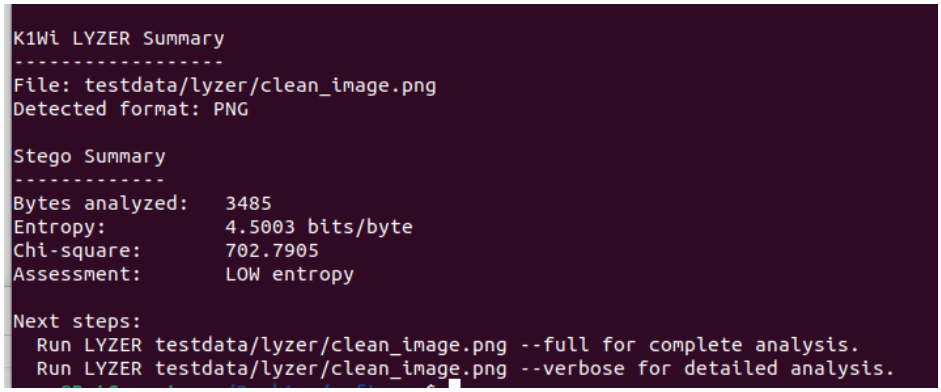
3.7 Perform a Forensic Image Scan

Command:

```
./bin/k1wi LYZER testdata/lyzer/clean_image.png --summary
```

Summary mode provides a quick triage view of the file, including format information, entropy, steganography indicators, assessment, and recommended next steps.

Figure 3.3 – LYZER Summary Mode



LYZER summary mode provides a concise forensic triage report that helps analysts quickly decide whether a file warrants deeper review.

For full analysis, use:

```
./bin/k1wi LYZER testdata/lyzer/clean_image.png --full
```

or:

```
./bin/k1wi LYZER testdata/lyzer/clean_image.png
```

Full analysis executes the available forensic modules, including entropy heatmap analysis, RS analysis, embedded signature detection, file carving, string intelligence, JPEG Huffman fingerprinting, and DCT coefficient analysis.

3.8 Analyze an ELF Binary

Command:

```
./bin/k1wi ELFINFO -IN ./bin/k1wi
```

K1Wi will display:

- ELF header information
- Program headers
- Dynamic libraries
- Symbol tables

This functionality is useful for reverse engineering, binary inspection, dependency review, and executable triage.

3.9 List PIE Symbols

Command:

```
./bin/k1wi PIECALC --bin ./bin/k1wi --list
```

This displays symbol names and offsets useful for:

- ASLR analysis
- PIE calculations
- CTF challenges
- Exploit development research
- Reverse engineering workflows

3.10 Analyze an RSA Challenge

Command:

```
./bin/k1wi RSA-FACTOR testdata/rsa/rsa_61_53_e17.txt
```

K1Wi will attempt to:

- Factor the modulus
- Recover the private exponent
- Decrypt the ciphertext
- Display recovered plaintext

This module is intended for educational cryptography and challenge-solving workflows. If classical factorization is unsuccessful, users can try additional RSA tools such as RSA-RHO, RSA-ECM, RSA-WIENER, RSA-SMALL-E, or RSA-KNOWNPQ depending on the structure of the challenge.

3.11 Execute the Regression Suite

Command:

```
./tests/run_regression.sh
```

Expected result:

```
PASS: 223
FAIL: 0
SKIP: 0
```

A successful run confirms that major framework functionality is operating correctly. Regression testing is especially useful after rebuilding K1Wi, switching branches, or making code changes.

3.12 Summary

This chapter demonstrated a basic K1Wi workflow: displaying help, analyzing strings, measuring entropy, performing image triage, inspecting ELF binaries, listing PIE symbols, testing RSA challenge input, and running the regression suite. The next chapter provides a detailed reference for the STRING analysis engine.

Chapter 4 – STRING Analysis Engine

4.1 Overview

The STRING module is K1Wi's universal string analysis engine. It is designed to identify, classify, and investigate textual data, encoded artifacts, and suspicious strings commonly encountered during cybersecurity investigations, digital forensics, reverse engineering, malware analysis, and Capture-The-Flag (CTF) challenges.

Unlike traditional string extraction utilities that simply display printable text, the STRING engine attempts to interpret the content being analyzed. It can identify common encodings, detect high-value artifacts, recognize suspicious patterns, and help analysts decide whether a string or file deserves deeper review.

STRING is often one of the first tools used during an investigation because suspicious content frequently appears as text, encoded data, credentials, URLs, email addresses, private-key markers, or fragments of structured data.

4.2 Basic Usage

Analyze a string directly:

```
./bin/k1wi STRING "hello world"
```

Analyze a file:

```
./bin/k1wi STRING --file  <file>
```

Decode detected content:

```
./bin/k1wi STRING --decode "<data>"
```

STRING can be used as a quick triage tool for individual strings or as part of a larger file-analysis workflow when reviewing extracted artifacts, carved files, archives, memory fragments, or challenge data.

4.3 Supported Detection Types

The STRING engine currently supports detection of:

- UTF-8 text
- Printable ASCII text
- Hexadecimal strings
- Shifted hexadecimal strings
- Base64 content
- URLs
- Email addresses
- JWT artifacts
- Credential-like strings
- Encoded private key markers

Additional detectors may be added in future releases as K1Wi's analysis capabilities expand.

4.4 Example 1 – Plain Text

Input:

```
./bin/k1wi STRING "Hello World!"
```

Example output:

```
Input: "Hello World!"
Length: 12
Printable: yes
Entropy: 3.0221
Detected Type: UTF-8 text
```

Interpretation:

The input appears to be normal human-readable text and does not exhibit characteristics commonly associated with encoded, encrypted, or suspicious content. This example demonstrates the baseline behavior of the STRING engine when processing ordinary printable text.

4.5 Example 2 – Shifted Hexadecimal Private Key Marker

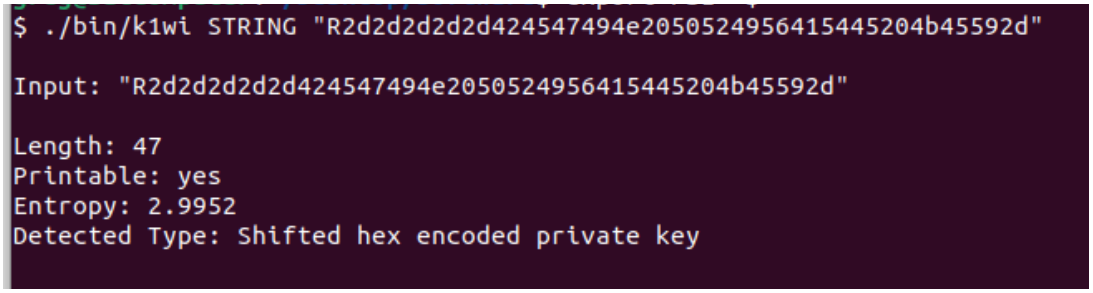
Input:

```
$ ./bin/k1wi STRING "R2d2d2d2d2d424547494e20505249564154415445204b45592d"
```

Example output:

```
Detected Type: Shifted hex encoded private key
```

Figure 4.1 – Shifted Hexadecimal Detection



The STRING engine identifies the input as a shifted hexadecimal encoded private key marker and reports the string length, printability, entropy, and detected type.

Interpretation:

The engine identified a shifted hexadecimal sequence that decodes to a recognizable private key marker. This capability can assist investigators in locating hidden cryptographic material embedded within files, memory dumps, archives, or challenge artifacts.

Shifted hexadecimal detection is useful because suspicious data is not always stored in a clean or obvious format. A marker may be offset, padded, shifted, partially encoded, or hidden inside a larger blob of data. STRING attempts to identify these patterns and surface them as high-value leads for further investigation.

4.6 Entropy Analysis

Every analyzed string receives an entropy measurement. Entropy is a statistical measure of randomness and can help analysts identify whether a string appears repetitive, human-readable, encoded, compressed, or potentially encrypted.

$$H = -\Sigma\ p_i\ \log_2\ p_i$$

General guidelines:

0–2	Highly repetitive
2–5	Typical text
5–7	Mixed, encoded, or partially structured data
7–8	Compressed, encrypted, or highly random data

Entropy should be treated as an indicator rather than proof. A high-entropy string may be encrypted, compressed, encoded, or simply random-looking data. A low-entropy string may still be meaningful if it contains credentials, commands, file paths, keys, or other useful artifacts.

4.7 String Intelligence

When analyzing files or extracted artifacts, K1Wi attempts to identify high-value textual indicators. Instead of only reporting that printable strings exist, the STRING engine looks for patterns that may be useful during investigation.

Examples include:

- Potential credentials
- Encoded secrets
- Private key indicators
- URLs
- Email addresses
- Suspicious encoded content
- Token-like or structured data

Findings are categorized according to severity and confidence when enough information is available. This helps analysts prioritize important leads instead of manually reviewing every printable string.

4.8 Limitations

The STRING module uses heuristic detection methods and may occasionally produce false positives or false negatives. Encoded data can appear in many formats, and not every suspicious-looking string is malicious or meaningful.

Users should validate significant findings through additional analysis. STRING results are best used as investigative leads that can be confirmed with other K1Wi modules, manual review, decoding tools, or external forensic utilities.

4.9 Summary

The STRING engine serves as K1Wi's primary text-analysis and string-triage module. It can identify ordinary text, detect common encodings, report entropy, and highlight suspicious or high-value textual artifacts.

In this chapter, STRING was used to analyze normal printable text, detect a shifted hexadecimal private key marker, and explain how entropy and string intelligence can support cybersecurity investigations, digital forensics, reverse engineering, and CTF workflows.

Chapter 5 – EXTRACT Engine

5.1 Overview

The EXTRACT engine is K1Wi's recursive extraction and file-carving framework. It is designed to process archives, containers, embedded files, and nested artifacts commonly encountered during digital forensic investigations, malware analysis, incident response, and Capture-The-Flag (CTF) challenges.

Unlike traditional extraction tools that stop after processing a single archive, the EXTRACT engine can recursively process newly discovered artifacts and continue analysis through multiple layers of nesting.

The goal of the EXTRACT engine is to automate artifact discovery while maintaining visibility into the extraction process. This makes EXTRACT useful when analyzing suspicious archives, multi-stage challenge files, malware droppers, embedded payloads, or forensic evidence that may contain hidden content.

5.2 Core Capabilities

The EXTRACT engine supports:

- Recursive extraction
- Archive processing
- Embedded artifact discovery
- File carving
- String intelligence integration
- Session tracking
- Branch tracking
- Extraction reporting

These capabilities allow K1Wi to move beyond simple archive extraction and operate as an artifact discovery workflow.

5.3 Basic Usage

Extract a file:

```
./bin/k1wi EXTRACT sample.zip
```

Extract a test archive included with the project:

```
./bin/k1wi EXTRACT testdata/archives/sample.zip
```

Perform recursive extraction:

```
./bin/k1wi EXTRACT --recursive testdata/archives/sample.zip
```

Recursive extraction instructs K1Wi to continue processing newly discovered files until no additional extractable content remains or the recursion limit is reached.

5.4 Extraction Sessions

Every extraction operation is assigned a unique session identifier.

Example:

```
EXTRACT SESSION 0001
```

The session number allows investigators to track results, correlate output directories, and maintain a record of extraction activity. This is useful when multiple extraction jobs are performed during the same investigation.

5.5 Extraction Workflow

The EXTRACT engine performs the following high-level workflow:

1. Validate the input file
2. Identify the file type
3. Extract supported content

4. Perform string intelligence analysis
5. Carve embedded artifacts
6. Queue newly discovered files
7. Repeat until completion

This workflow allows K1Wi to process nested structures automatically while still showing the analyst where extracted and carved artifacts are written.

5.6 Recursive Extraction

Recursive mode is one of the most powerful capabilities of the EXTRACT engine.

Example:

```
./bin/k1wi EXTRACT --recursive suspicious_archive.zip
```

Possible structure:

```
archive.zip
 ├── image.jpg
 ├── document.pdf
 └── hidden.zip
     ├── notes.txt
     └── payload.bin
```

When recursion is enabled, K1Wi continues processing newly discovered archives and artifacts until the extraction tree has been exhausted or the configured safety limit is reached.

5.7 Output Directories

Example:

```
carved/extract_0001/
```

Within this directory, K1Wi stores:

- Extracted files
- Carved artifacts
- Intermediate processing results
- Final output artifacts

The directory structure preserves the extraction path and investigation history. This helps analysts understand where files came from and how they were discovered.

5.8 Branch Tracking

K1Wi tracks extraction branches to help investigators understand artifact lineage.

Example:

```
depth_00_branch_000_final
```

This naming convention allows users to determine:

- Recursion depth
- Branch origin
- Final artifact location

Branch tracking is especially useful when processing large or heavily nested archives because it helps show how each recovered file relates to the original input.

5.9 String Intelligence Integration

During extraction, K1Wi automatically invokes STRING analysis on discovered artifacts.

Example output:

```
[STRING] file=sample.zip ASCII=5 UTF8=0 Base64=0 URL=0 Email=0
```

This provides investigators with immediate visibility into potentially useful content without requiring separate analysis steps. STRING integration can help identify printable text, encoded data, URLs, email addresses, and other useful indicators during the extraction process.

5.10 File Carving

The EXTRACT engine can identify and recover embedded content discovered during analysis.

Potential examples include:

- ZIP archives
- JPEG images
- PDF documents
- PNG images
- GZIP streams

Recovered artifacts are automatically added to the extraction workflow when appropriate. This allows K1Wi to continue analyzing content that may not be visible through normal archive extraction alone.

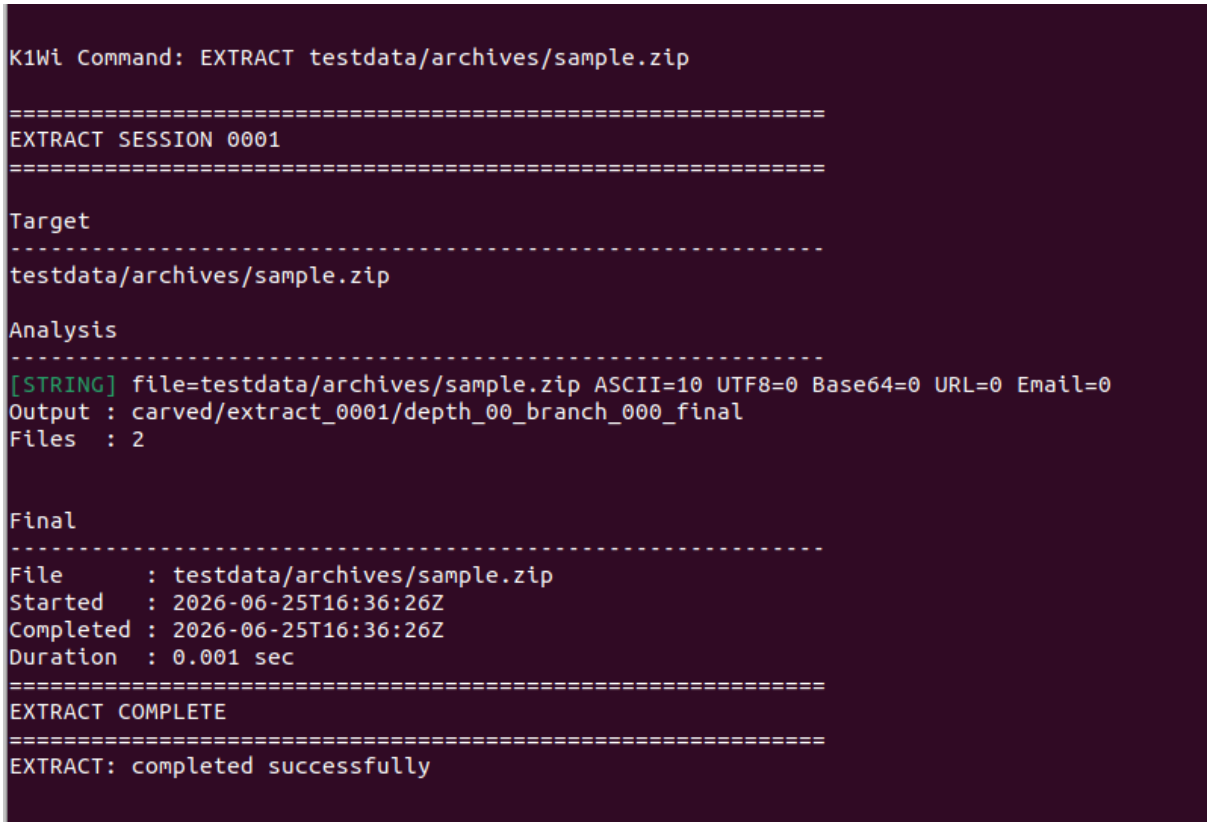
5.11 Example Session

Input:

```
./bin/k1wi EXTRACT testdata/archives/sample.zip
```

Example output:

Figure 5.1 – EXTRACT Session Output



The EXTRACT engine creates an extraction session, analyzes the target archive, performs string intelligence, writes recovered files to a carved output directory, and reports completion status.

In this example, K1Wi processes the included sample ZIP archive and stores the recovered output under a session-specific directory. The output shows the target file, analysis results, carved output path, recovered file count, timing information, and completion status.

5.12 Common Use Cases

The EXTRACT engine is useful for:

- CTF archive challenges
- Malware dropper analysis
- Suspicious attachment review
- Nested archive investigation
- Embedded payload discovery
- File-carving workflows
- Digital forensic triage

Because EXTRACT combines archive processing, recursive analysis, string intelligence, and carving, it can reduce the amount of manual work required when investigating layered or suspicious files.

5.13 Limitations

The EXTRACT engine is designed for triage and artifact discovery. It may not support every archive format, compression method, encryption scheme, or malformed file structure.

Password-protected archives, intentionally corrupted files, unsupported containers, or highly obfuscated payloads may require additional tools or manual analysis. Users should treat EXTRACT results as investigative output and validate important findings before drawing conclusions.

5.14 Summary

The EXTRACT engine provides K1Wi with recursive extraction, file carving, session tracking, branch tracking, and integrated string intelligence. It helps users discover hidden or nested artifacts while preserving useful context about how each artifact was recovered.

In this chapter, EXTRACT was used to process a sample archive, demonstrate session-based output, explain recursive extraction, and show how K1Wi organizes recovered files for later analysis.

Chapter 6 – COPY Verified File Copy

6.1 Overview

The COPY command is K1Wi's forensic-style verified file copy utility. It is designed to copy a regular file from one location to another while confirming that the destination file matches the source file after the copy completes.

Unlike a basic copy operation, COPY performs integrity verification after writing the destination file. This makes it useful during forensic, testing, evidence-handling, and controlled analysis workflows where the analyst needs confirmation that a copied artifact was not changed during the copy process.

COPY is intended for verified local file copying. Remote SSH transfer is not currently part of COPY and may be considered separately in a future release.

6.2 Basic Usage

Copy a file:

```
COPY <source> <destination>
```

Overwrite an existing destination intentionally:

```
COPY <source> <destination> --force
```

By default, COPY refuses to overwrite an existing destination file. This prevents accidental replacement of files during investigations or testing. The --force option must be supplied when overwrite behavior is intentional.

6.3 Verification Behavior

COPY verifies the copied file using:

- Source and destination file size comparison
- Source and destination SHA-256 comparison
- Source and destination MD5 comparison

SHA-256 is the primary integrity hash. MD5 is also reported for compatibility with legacy forensic workflows, existing evidence records, and tools that still use MD5 for file identification.

A successful copy prints a COPY Verification Report showing the source and destination paths, copied byte count, source and destination file sizes, SHA-256 values, MD5 values, match status, and final verification result.

6.4 Safety Behavior

COPY includes several safety checks:

- Rejects missing source files
- Rejects directory sources
- Rejects same source and destination paths
- Rejects unknown command options
- Refuses overwrite unless --force is supplied
- Reports verification failure if size, SHA-256, or MD5 values do not match

These checks help prevent accidental overwrite, incorrect source selection, and silent copy failures.

6.5 Example Session

Input:

```
COPY testdata/text/hello.txt /tmp/k1wi_copy_test.txt
```

Example successful output:

```
COPY Verification Report
------------------------
Source       : testdata/text/hello.txt
Destination  : /tmp/k1wi_copy_test.txt
Bytes copied : 47
Source size  : 47
Dest size    : 47
Source SHA256: ...
Dest SHA256  : ...
Source MD5   : ...
Dest MD5     : ...
Size match   : yes
SHA256 match : yes
MD5 match    : yes
Verification : PASS
```

When verification succeeds, COPY reports PASS and confirms that the copied file matches the source according to size, SHA-256, and MD5 comparison.

6.6 Common Use Cases

COPY is useful for:

- Verified local file copying
- Forensic artifact handling

- Evidence-handling workflows
- Test file preservation
- Copying recovered files for later analysis
- Confirming copied files match original sources
- Maintaining hash records during controlled analysis

Because COPY reports both SHA-256 and MD5 values, it can support modern integrity verification while remaining compatible with older forensic workflows that still record MD5 hashes.

6.7 Limitations

COPY is designed for local verified file copying. It does not currently perform remote SSH transfer, network copy operations, encrypted transport, or multi-destination evidence replication.

COPY verifies that the destination file matches the source after the copy completes, but users should still follow their organization’s evidence-handling, chain-of-custody, storage, and retention procedures when working with forensic material.

6.8 Summary

The COPY command provides K1Wi with a verified local file-copy workflow. By comparing file size, SHA-256, and MD5 values, COPY helps confirm that a copied artifact matches the original source.

In this chapter, COPY was used to perform a verified copy, explain overwrite protection, describe safety checks, and show how K1Wi reports file integrity after copying. The next chapter introduces the LYZER forensic analysis engine.

Chapter 7 – LYZER Forensic Analysis Engine

7.1 Overview

LYZER is K1Wi's forensic image-analysis engine. It is designed to help analysts identify embedded content, suspicious artifacts, steganography indicators, hidden data, and high-value textual findings within image files.

LYZER combines multiple forensic techniques into a single workflow, including entropy analysis, entropy heatmap generation, string extraction, embedded signature detection, file carving, steganography heuristics, JPEG structure inspection, Huffman fingerprinting, DCT coefficient analysis, and string intelligence.

The goal of LYZER is not to definitively prove that a file contains hidden information. Instead, LYZER identifies indicators that may warrant further investigation. By combining multiple analysis methods, the engine provides analysts with a broader view of the file and helps prioritize additional forensic examination.

7.2 Core Analysis Modules

The LYZER engine currently includes the following analysis modules:

Entropy Analysis
Measures overall file randomness and helps identify compressed, encrypted, or unusual data regions.

Entropy Heatmap
Breaks the file into blocks and highlights localized entropy anomalies.

RS Analysis
Evaluates least-significant-bit behavior and provides steganography indicators.

Embedded Signature Detection
Searches for known file signatures such as ZIP, PDF, JPEG, PNG, GZIP, and executable headers.

File Carving
Attempts to recover embedded artifacts discovered inside the analyzed file.

String Intelligence
Identifies useful textual artifacts, credentials, URLs, email addresses, encoded content, and suspicious strings.

JPEG Huffman Fingerprinting
Examines JPEG Huffman tables and highlights non-standard compression characteristics.

DCT Analysis
Evaluates JPEG coefficient distributions for signs of recompression, manipulation, or unusual encoding behavior.

7.2.1 Output Modes Added in v1.2

K1Wi v1.2 expands LYZER with several output modes so users can choose between quick triage, quiet reports, full forensic output, and machine-readable JSON.

Basic usage:

```
LYZER <file>
LYZER <file> --summary
LYZER <file> --quiet
LYZER <file> --json
LYZER <file> --full
LYZER <file> --verbose
LYZER <file> ALL
```

Mode descriptions:

- --summary produces a short report with file format, entropy, steganography summary, assessment, and next steps.
- --quiet produces a minimal analyst-friendly report with assessment and recommended follow-up.
- --json produces a machine-readable summary suitable for scripting and regression checks.
- --full and --verbose run the complete forensic workflow, including carving and string intelligence.
- ALL remains supported as a full-analysis mode for compatibility with earlier workflows.

The default LYZER behavior favors summary-style output so routine triage is easier to read. Analysts can still request the full forensic view whenever detailed evidence is required.

7.3 Understanding LYZER Results

LYZER findings should be treated as investigative indicators rather than final conclusions. A single anomaly is rarely enough to prove that a file contains hidden data. Multiple independent findings generally increase confidence and justify deeper analysis.

Entropy interpretation:

0.0–2.0 Repetitive or mostly empty data.

2.0–5.0 Typical text or structured content.

5.0–7.0 Mixed content, moderate compression, or partially structured data.

7.0–8.0 Highly compressed, encrypted, or random-looking data.

8.0 Maximum randomness.

RS analysis interpretation:

Balanced R/S values Typical image behavior.

Significant R/S deviation Potential steganography indicator.

Large flipped-group changes Possible least-significant-bit embedding.

Multiple anomalies Increased suspicion.

Normal RS profile No strong steganography indicator detected.

7.4 Baseline Image Analysis

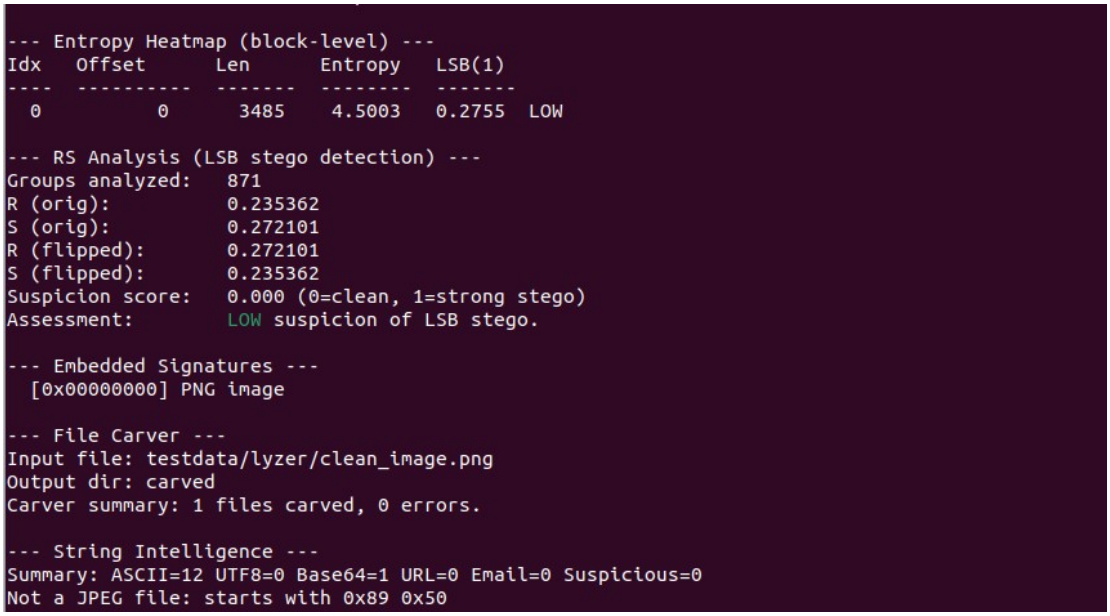
The first example uses clean_image.png to establish a baseline for comparison. A clean image should contain no embedded archives, hidden payloads, suspicious strings, or unusual artifacts beyond what would normally be expected in an image file.

When analyzing a clean image, LYZER reports the detected file format, file size, entropy measurements, embedded signatures, carving results, and string intelligence findings. Entropy values are useful for identifying compressed or encrypted regions of data, while string extraction provides visibility into human-readable content contained within the file.

A baseline analysis helps analysts understand what normal output looks like before investigating more complex files that may contain hidden information.

7.4.1 Example 1 – Baseline Image Analysis

Figure 7.1 – Baseline Image Analysis



Analyst Interpretation

The image was successfully identified as a PNG file. Entropy values were within expected ranges, no embedded files were detected, and String Intelligence reported no significant high-value findings. This output establishes a baseline for comparison with more suspicious files.

7.5 Hidden Text and Secret Detection

The second example uses k1wi_demo.jpg, which contains intentionally appended text data for demonstration purposes. Although the image appears normal when opened in a standard image viewer, additional information exists beyond the visible image content.

LYZER identifies indicators of suspicious content through its String Intelligence subsystem. During analysis, the module detected a high-value finding in the form of an embedded API key:

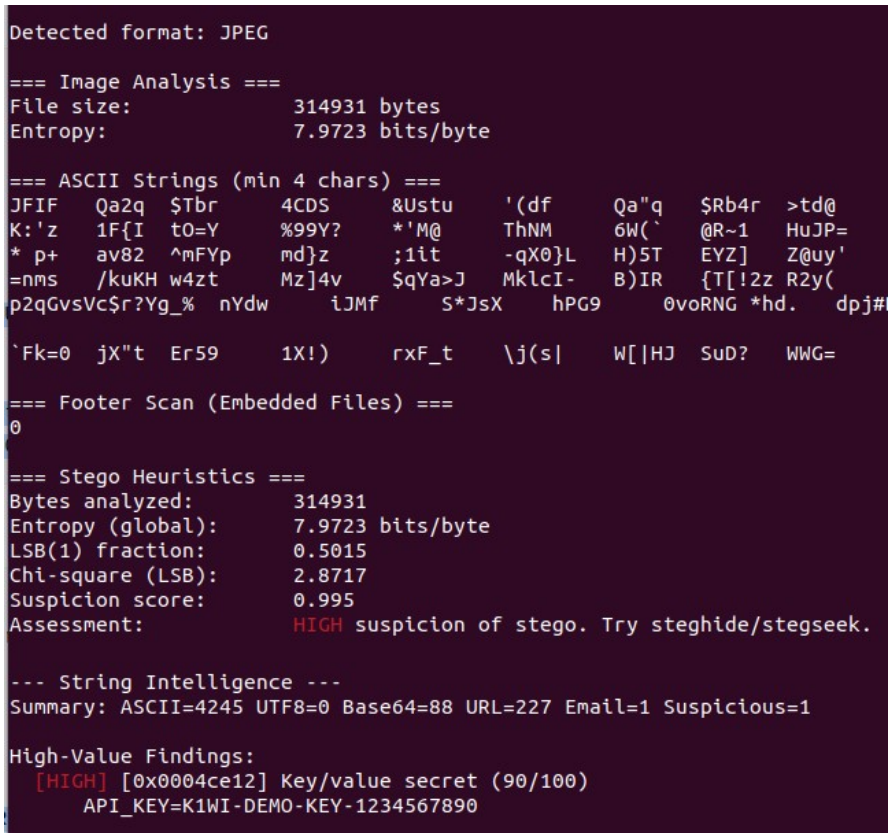
```
API_KEY=K1WI-DEMO-KEY-1234567890
```

This finding demonstrates how LYZER prioritizes potentially sensitive information rather than simply displaying large quantities of extracted text. The String Intelligence engine attempts to identify credentials, secrets, tokens, URLs, email addresses, and other high-value artifacts that may be useful during a forensic investigation.

The analysis also reported elevated steganography suspicion scores. Such findings do not prove the existence of hidden data, but they provide useful indicators that further examination may be warranted.

7.5.1 Example 2 – Hidden Text Detection

Figure 7.2 – Analysis of k1wi_demo.jpg



LYZER identified elevated steganography indicators and detected a high-value finding containing an embedded API key within the image.

Analyst Interpretation

Although the image appears normal when viewed with a standard image viewer, LYZER identified a hidden text payload containing an API key. The String Intelligence subsystem automatically prioritized the artifact as a high-value finding, allowing the investigator to quickly identify potentially sensitive information without manually reviewing large amounts of extracted text.

7.6 Embedded Archive Detection and File Carving

The final example uses embedded_zip.jpg, a JPEG image containing an appended ZIP archive. Although the image remains viewable as a standard JPEG, additional data exists beyond the normal end of the image content.

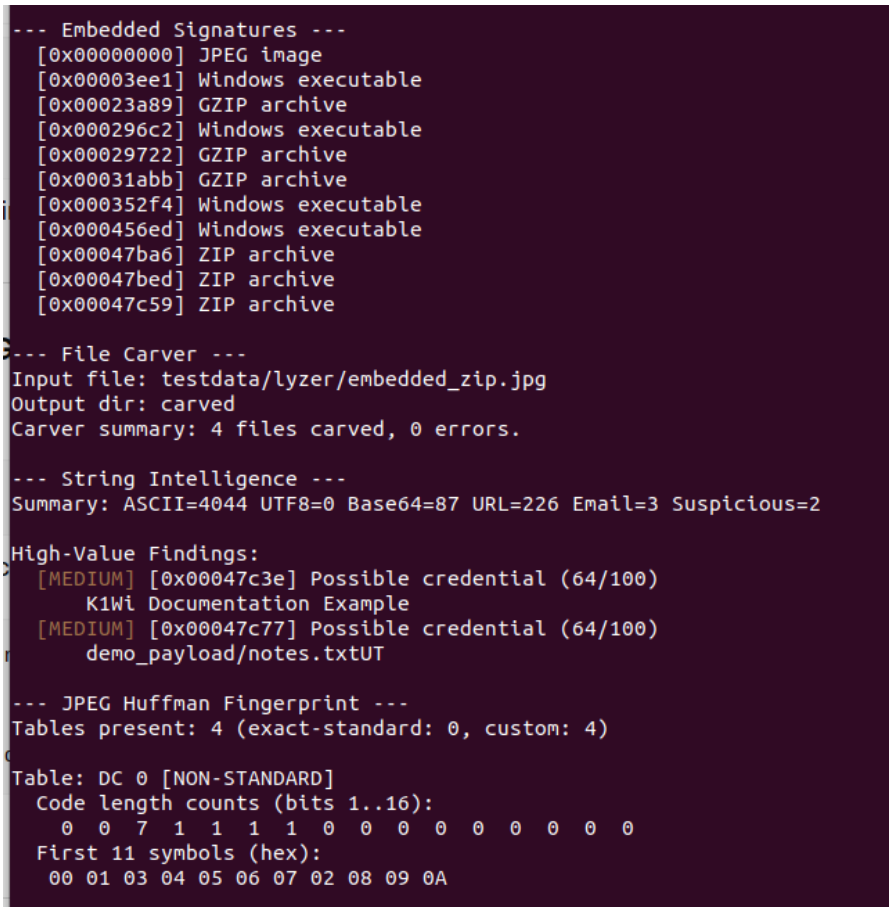
During analysis, LYZER detects embedded signatures associated with archive formats and invokes the file-carving subsystem. The file carver extracts recoverable content and writes recovered files to a dedicated output directory for additional examination.

This capability is useful when investigating files that may contain appended archives, hidden payloads, malware droppers, or other embedded content designed to evade casual inspection.

Unlike simple string extraction, file carving attempts to recover complete embedded objects. This allows analysts to continue examining recovered files with other K1Wi modules.

7.6.1 Example 3 – Embedded Archive Recovery

Figure 7.3 – Embedded Archive Recovery from embedded_zip.jpg



LYZER detected embedded archive signatures, carved recoverable files, and reported high-value findings associated with the recovered content.

Analyst Interpretation

Although embedded_zip.jpg appears to be a normal JPEG image when viewed with standard image-viewing software, LYZER identified an embedded ZIP archive contained within the file. The Embedded Signature Detection module located the archive, and the File Carver successfully recovered the embedded content for further examination.

This type of finding is significant because archives can conceal documents, payloads, malware, or other artifacts that are not visible during normal file viewing. The presence of a recoverable archive increases the investigative priority of the file and warrants additional analysis of the recovered contents.

7.7 Advanced Analysis Modules

7.7.1 Embedded Signature Detection

The Embedded Signature Detection module searches files for known file headers and signatures. This capability helps identify hidden archives, documents, executables, and other embedded content that may not be visible

through normal file viewing. When a valid signature is detected, investigators can use the File Carver module to recover the embedded object.

Common signature interpretations:

ZIP Possible embedded archive.

PDF Possible embedded document.

EXE Possible embedded executable.

JPEG / PNG Possible embedded image.

Unknown Manual review may be required.

7.7.2 JPEG Huffman Fingerprinting

JPEG Huffman Fingerprinting examines the Huffman tables used to compress JPEG images. Standard tables are commonly produced by digital cameras and image-editing software, while custom tables may indicate recompression, specialized software, image manipulation, or intentionally modified encoding.

Common Huffman fingerprint interpretations:

Standard
Typical JPEG compression behavior.

Mixed
Possible modified or recompressed image.

Custom
Possible custom encoder or non-standard compression.

Multiple Custom
Increased suspicion and recommended manual review.

7.7.3 DCT Coefficient Analysis

DCT Coefficient Analysis evaluates the distribution of JPEG compression coefficients. Unusual coefficient patterns may indicate image manipulation, recompression, or potential steganographic activity. Because legitimate image processing can also affect coefficient distributions, DCT analysis should be considered a supporting indicator rather than definitive proof of hidden content.

Common DCT interpretations:

Natural
Typical JPEG coefficient distribution.

Minor Deviations
Possible recompression or normal editing artifacts.

Significant Deviations
Further review recommended.

Extreme Deviations
Possible manipulation or unusual encoding.

7.7.4 Confidence Levels

LYZER findings may be assigned confidence levels to help analysts prioritize review.

LOW
Archive the result or use it as baseline context.

MEDIUM
Review manually and compare against other findings.

HIGH
Investigate further with supporting tools.

CRITICAL
Perform immediate analyst review.

7.7.5 Recommended Investigator Actions

High Entropy
Review the entropy heatmap and compare against file structure.

RS Anomaly
Perform additional steganography testing.

Embedded Signature Found
Run or review file-carving output.

Secret Found
Investigate the recovered secret or credential-like artifact.

Archive Found
Analyze recovered files and inspect extraction output.

7.8 Interpreting Results

No single indicator should be considered proof of malicious activity. Analysts should evaluate entropy measurements, string intelligence findings, embedded signatures, file-carving results, and steganography heuristics collectively.

For example, a file may exhibit high entropy because it contains compressed image data rather than hidden information. Likewise, a steganography heuristic may generate a false positive on a legitimate image. LYZER is designed to provide investigative leads and indicators rather than definitive conclusions.

The most effective approach is to combine multiple findings and follow up with additional analysis when suspicious artifacts are identified.

7.9 Summary

LYZER provides a unified workflow for image-based forensic analysis. Through entropy measurements, string intelligence, embedded signature detection, file carving, steganography analysis, and JPEG structure inspection, the module helps analysts identify potentially hidden information that may not be visible through conventional image-viewing tools.

The examples presented in this chapter demonstrate three common scenarios:

1. Baseline analysis of a normal image.
2. Detection of hidden text and sensitive information.
3. Recovery of embedded archives through file carving.

Together, these examples illustrate how LYZER can assist analysts in identifying suspicious content and prioritizing further forensic investigation.

Chapter 8 – PIECALC Engine

8.1 Overview

PIECALC is K1Wi's Position Independent Executable address-calculation and symbol-resolution engine. It is designed to assist analysts working with Linux ELF binaries that use Position Independent Executable (PIE) behavior and Address Space Layout Randomization (ASLR).

Modern Linux executables are often loaded at randomized base addresses each time they run. This means that addresses observed during debugging, crash analysis, exploit development research, or reverse engineering may not directly match the static offsets shown in the binary.

PIECALC helps bridge that gap by listing symbols, resolving offsets, calculating runtime addresses, and identifying the nearest known symbol to a supplied address.

8.2 Understanding PIE Binaries

Position Independent Executables do not execute at fixed memory addresses. Instead, the operating system loads the binary at a randomized base address during program startup. This behavior improves security by making memory addresses less predictable.

For analysts, this means that a leaked runtime address often needs to be translated back into a file offset or known function symbol. PIECALC automates this process by combining ELF symbol information with user-supplied addresses.

This makes PIECALC useful for:

- Reverse engineering
- Debugging
- Crash analysis
- ASLR research
- CTF challenge solving
- Exploit development education
- Binary triage

PIECALC is intended for authorized analysis, education, research, and defensive workflows.

8.3 Core Features

PIECALC provides the following core features:

Symbol Listing
Displays available function symbols and offsets from the target ELF binary.

Nearest Symbol Lookup
Identifies the closest known symbol to a supplied address.

Address Resolution
Helps translate between observed addresses, offsets, and known program locations.

JSON Output
Produces machine-readable output suitable for scripting, automation, and regression testing.

ELF Integration
Works directly with ELF symbol information discovered from the analyzed binary.

8.4 Basic Usage

List symbols from a binary:

```
./bin/k1wi PIECALC --bin ./bin/k1wi --list
```

Find the nearest known symbol to an address:

```
./bin/k1wi PIECALC --bin ./bin/k1wi --nearest 0xdce0
```

Generate JSON output:

```
./bin/k1wi PIECALC --bin ./bin/k1wi --json --list
```

PIECALC can also be used from inside the interactive K1Wi shell by entering the command without the leading executable path:

```
PIECALC --bin ./bin/k1wi --list
```

8.5 Example 1 – Listing Symbols

Command:

```
./bin/k1wi PIECALC --bin ./bin/k1wi --list
```

Purpose:

The --list option displays symbols discovered within the target binary. Symbol listings provide analysts with a quick overview of available functions and offsets. This is useful when examining program structure, identifying candidate functions, reviewing module names, or preparing for deeper reverse engineering work.

Figure 8.1 – PIECALC Symbol Enumeration

PIECALC lists function symbols and offsets recovered from the K1Wi executable.

Analyst Interpretation:

PIECALC can enumerate function symbols contained within an ELF executable and display their corresponding offsets. This capability allows analysts to quickly identify application routines, cryptographic functions, forensic modules, and helper functions. Symbol enumeration is useful during reverse engineering, exploit development education, binary triage, and debugging because it provides a map of executable functionality without requiring manual inspection of the binary.

8.6 Example 2 – Nearest Symbol Lookup

Command:

```
./bin/k1wi PIECALC --bin ./bin/k1wi --nearest 0xdce0
```

Purpose:

The --nearest option identifies the closest known symbol to a supplied address. This is useful when analyzing crash reports, debugger output, leaked addresses, or offsets observed during reverse engineering.

Figure 8.2 – PIECALC Nearest Symbol Lookup

PIECALC identifies the nearest known symbol to address 0xdce0.

```
K1Wi Command: PIECALC --bin ./bin/k1wi --nearest 0xdce0
[NEAREST SYMBOLS]
  handle_file_panel_key    off=0xdca0 dist=0x40
  draw_panel_settings      off=0xdc20 dist=0xc0
  draw_panel_view_plaintext off=0xd690 dist=0x650
  draw_panel_vigauto       off=0xd610 dist=0x6d0
  draw_panel_vigrefine     off=0xd590 dist=0x750
```

Analyst Interpretation:

Addresses observed during execution do not always correspond exactly to the beginning of a function. Nearest-symbol lookup provides context by identifying the closest known function and its offset relationship to the supplied address. This helps analysts determine which part of the program is associated with the observed address.

In this example, PIECALC identifies the nearest known symbol to the supplied address and reports nearby symbol context. This allows analysts to quickly understand the surrounding program structure.

8.7 Example 3 – JSON Output

Command:

```
./bin/k1wi PIECALC --bin ./bin/k1wi --json --list
```

Purpose:

The --json option produces machine-readable output that can be used by scripts, automated workflows, regression tests, or external analysis tools.

Figure 8.3 – PIECALC JSON Output

PIECALC produces structured JSON output for symbol enumeration.

Analyst Interpretation:

JSON output is useful when PIECALC results need to be consumed by another tool or stored as structured evidence. Instead of manually parsing terminal text, scripts can read the JSON fields directly. This makes PIECALC easier to integrate into repeatable reverse engineering workflows, automated testing, and analysis pipelines.

Table 8.2 – PIECALC Output Fields

Field	Description
Symbol Name	Name of the discovered function
Offset	Relative offset within the binary
Runtime Address	Calculated runtime address
Distance	Difference between target address and nearest symbol
Binary	Target executable being analyzed
Matches	Symbols returned by the query

8.8 Practical Reverse Engineering Workflow

A typical PIECALC workflow may include the following steps:

1. Identify the target ELF binary.
2. List available symbols using --list.
3. Observe a runtime address through debugging, crash output, or controlled analysis.
4. Use --nearest to identify the closest known function.
5. Compare the result against surrounding symbols and offsets.
6. Use JSON output if the results need to be scripted or documented.

Example workflow:

```
./bin/k1wi PIECALC --bin ./bin/k1wi --list
./bin/k1wi PIECALC --bin ./bin/k1wi --nearest 0xdce0
./bin/k1wi PIECALC --bin ./bin/k1wi --json --list
```

This workflow helps analysts move from raw addresses to meaningful program context.

8.9 Summary

PIECALC provides K1Wi with practical reverse engineering support for PIE-enabled ELF binaries. It can list symbols, resolve nearby functions, assist with address interpretation, and produce JSON output for automated workflows.

In this chapter, PIECALC was used to enumerate symbols, locate the nearest known symbol to a supplied address, and generate machine-readable JSON output. These capabilities make PIECALC useful for binary triage, debugging, reverse engineering, CTF challenge solving, and educational exploit-development research.

Chapter 9 – ELFINFO Engine

9.1 Overview

ELFINFO is K1Wi's ELF binary-inspection utility. It allows analysts to examine executable files, shared libraries, and other ELF-based binaries without relying on external tools such as readelf or objdump.

ELFINFO extracts information directly from ELF headers and related binary structures to provide insight into architecture, file type, entry point, program layout, imported libraries, and symbol information. This capability is useful during malware analysis, reverse engineering, vulnerability research, exploit-development education, dependency review, and general binary inspection.

The command supports ELF analysis workflows for Linux binaries and presents information in a concise analyst-friendly format.

9.2 Understanding ELF Files

The Executable and Linkable Format, commonly called ELF, is the standard binary format used by Linux and many Unix-like operating systems. ELF files may contain executable code, metadata, memory layout information, imported libraries, debugging information, relocation data, and symbol tables.

When a program is executed, the operating system uses information stored inside the ELF file to determine how the binary should be loaded into memory.

Key ELF structures include:

- ELF header
- Program headers
- Section headers
- Dynamic linking information
- Symbol tables

ELFINFO provides visibility into these structures so analysts can understand how a binary is organized and how it may behave at runtime.

9.3 Core Features

ELFINFO provides the following capabilities:

ELF Header Analysis
Displays architecture, file type, entry point, class, endianness, and other high-level binary metadata.

Program Header Enumeration
Shows loadable segments and memory-layout information used by the operating system loader.

Section Header Review
Provides visibility into named sections such as code, data, symbol, relocation, and metadata sections.

Dynamic Library Identification
Lists imported shared libraries required by the binary at runtime.

Symbol Table Extraction
Displays available symbols that can assist with reverse engineering and binary triage.

32-bit and 64-bit ELF Awareness
Supports inspection of common ELF binary classes used on Linux systems.

Dependency Analysis
Helps identify required runtime libraries and external dependencies.

Binary Reconnaissance
Provides a quick first-pass view of executable structure before deeper analysis.

9.4 Basic Usage

Command:

```
./bin/k1wi ELFINFO -IN <binary>
```

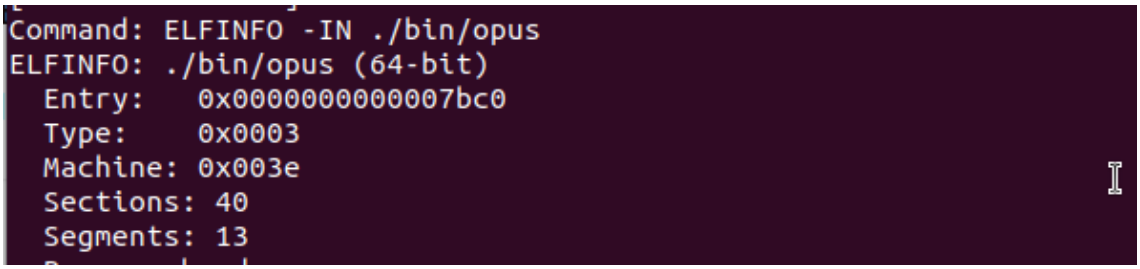
Example:

```
./bin/k1wi ELFINFO -IN ./bin/k1wi
```

Purpose:

The ELFINFO command reads and displays structural information from the specified ELF binary. This information can help identify architecture details, imported libraries, memory layout information, and available symbols.

Figure 9.1 – Basic ELFINFO Analysis



ELFINFO displays architecture, entry point, file type, and structural information for the K1Wi executable.

Analyst Interpretation:

The ELF header provides fundamental information about a binary file. This includes the executable type, target architecture, entry point address, section count, and program-header count. Analysts frequently begin ELF investigations by reviewing the ELF header because it provides a high-level overview of how the binary is organized.

In this example, ELFINFO identifies the K1Wi executable as an ELF binary and displays its entry point and structural characteristics.

9.5 Example 1 – Dynamic Library Enumeration

Command:

```
./bin/k1wi ELFINFO -IN ./bin/k1wi
```

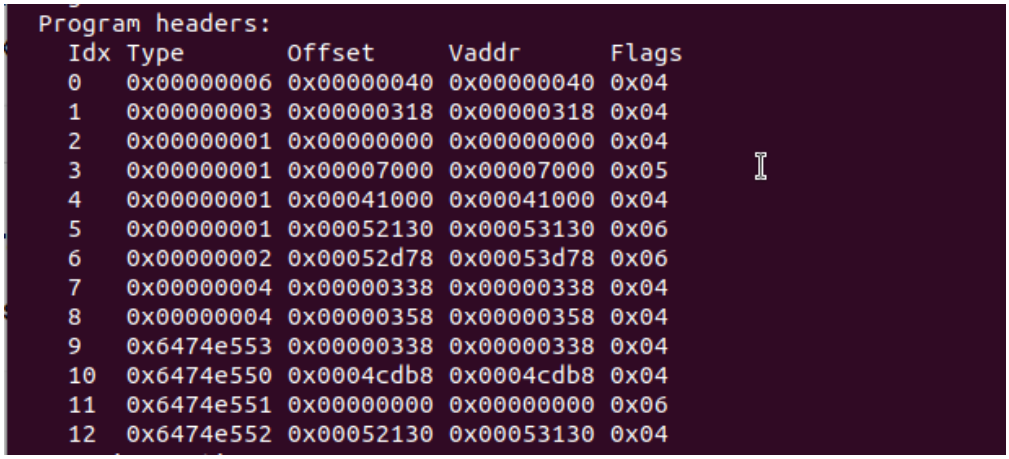
Purpose:

Dynamic library enumeration shows which shared libraries the executable expects to load at runtime. These dependencies can reveal important information about what capabilities the binary may use.

Examples may include libraries for:

- Cryptography
- Compression
- Image processing
- Terminal interfaces
- Large integer arithmetic
- Standard C runtime support

Figure 9.2 – ELFINFO Dynamic Library Enumeration



ELFINFO lists imported shared libraries used by the K1Wi executable.

Analyst Interpretation:

Imported libraries provide useful context about a binary's functionality. For example, a dependency on cryptographic libraries may suggest hashing, encryption, or certificate-related behavior. A dependency on image-processing libraries may suggest support for image parsing or forensic analysis. A dependency on terminal-interface libraries may indicate interactive terminal features.

In this example, ELFINFO helps identify the runtime dependencies used by K1Wi. This gives analysts a quick way to understand which external libraries the executable relies on before performing deeper reverse engineering.

9.6 Example 2 – Symbol Table Analysis

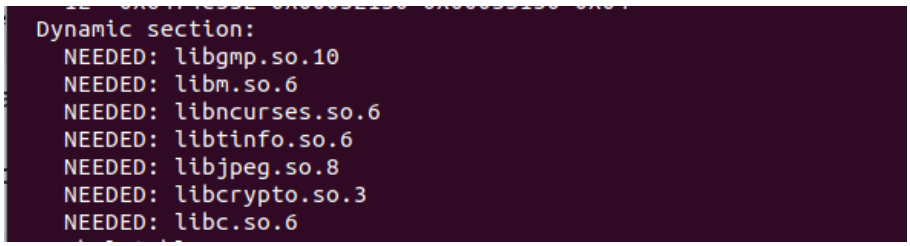
Command:

```
./bin/k1wi ELFINFO -IN ./bin/k1wi
```

Purpose:

Symbol table analysis displays function names, object names, and other symbol metadata when symbols are available in the binary. Symbols can help analysts understand program structure without fully disassembling the binary.

Figure 9.3 – ELFINFO Symbol Table Analysis



ELFINFO displays symbol information recovered from the K1Wi executable.

Analyst Interpretation:

Symbols are valuable during reverse engineering because they provide names for functions, global objects, and internal routines. A stripped binary may expose few or no useful symbols, while a development or debug build may provide many symbol names.

In this example, ELFINFO identifies available symbols within the K1Wi executable. These names can help analysts quickly locate command handlers, analysis modules, cryptographic routines, and other important parts of the program.

9.7 Practical ELF Analysis Workflow

A practical ELFINFO workflow may include the following steps:

1. Identify the target binary.
2. Run ELFINFO to review the ELF header.
3. Review program headers and memory-layout information.
4. Identify imported shared libraries.
5. Review available symbols.
6. Use PIECALC when address translation or symbol-offset analysis is needed.
7. Continue deeper analysis with debuggers, disassemblers, or external forensic tools when necessary.

Example workflow:

```
./bin/k1wi ELFINFO -IN ./bin/k1wi
./bin/k1wi PIECALC --bin ./bin/k1wi --list
```

ELFINFO is often used before PIECALC because it provides the initial binary overview, while PIECALC helps translate symbol offsets and runtime addresses.

9.8 Summary

ELFINFO provides K1Wi with practical ELF binary-inspection capabilities. It can display ELF headers, program structure, imported libraries, and symbol information in a concise format useful for analysts.

In this chapter, ELFINFO was used to inspect the K1Wi executable, review dynamic library dependencies, and analyze available symbol information. These capabilities make ELFINFO useful for binary triage, dependency review, reverse engineering, malware analysis, vulnerability research, CTF workflows, and educational binary-analysis tasks.

Chapter 10 – RSA Analysis Tools

10.1 Overview

The RSA Analysis Tool Suite provides K1Wi with educational and investigative modules for analyzing RSA cryptographic challenges. These tools are designed to assist cybersecurity students, security researchers, Capture-The-Flag (CTF) participants, and reverse engineers when examining RSA parameters, recovering weak keys, validating factors, and solving cryptographic exercises.

Unlike general-purpose cryptographic libraries, the K1Wi RSA modules focus on transparency and education. The tools expose intermediate values, recovery steps, and analyst-relevant output so users can better understand how RSA weaknesses and key-recovery techniques operate.

The RSA tool suite currently includes:

- RSA-FACTOR – Classical and Fermat factorization
- RSA-RHO – Pollard Rho factorization
- RSA-ECM – Elliptic Curve Method factorization
- RSA-WIENER – Wiener's continued fraction attack
- RSA-SMALL-E – Low public exponent attack
- RSA-KNOWNPQ – Decrypt using known prime factors
- RSA-CHECKPQ – Validate candidate RSA primes
- RSA-DFROMPQ – Recover the private exponent
- RSA-MINI – Educational RSA challenge helper

These tools are intended for educational use, challenge solving, cryptographic research, and controlled security testing environments.

10.2 Understanding RSA

RSA is a public-key cryptographic system based on the mathematical difficulty of factoring large composite integers. An RSA key pair is built from several important values.

Core RSA values:

```
p  The first prime factor.
q  The second prime factor.
n  The RSA modulus, calculated as n = p × q.
e  The public exponent.
d  The private exponent.
```

The public key contains n and e. The private key contains d, which is derived from the prime factors and Euler's Totient Function.

Euler's Totient Function:

$$\varphi(n) = (p - 1)(q - 1)$$

The private exponent is calculated as the modular inverse of e modulo φ(n):

$$d \equiv e^{-1} \bmod \varphi(n)$$

Encryption:

$$c = m^e \bmod n$$

Decryption:

$$m = c^d \bmod n$$

Many RSA attacks rely on weaknesses in key generation, unusually small parameters, known factors, poor exponent choices, or implementation mistakes. The K1Wi RSA modules demonstrate several of these techniques in a controlled educational setting.

10.3 RSA-FACTOR

Purpose:

RSA-FACTOR performs classical RSA modulus factorization using Fermat and related factorization techniques. When successful, the tool recovers the prime factors of the modulus, computes the private exponent, and attempts plaintext recovery.

This module is useful for educational RSA challenges, weak-key demonstrations, and CTF exercises where the RSA modulus is intentionally vulnerable.

Usage:

```
./bin/k1wi RSA-FACTOR <rsa_file>
```

Example:

```
./bin/k1wi RSA-FACTOR testdata/rsa/rsa_61_53_e17.txt
```

Figure 10.1 – RSA-FACTOR Recovery

RSA-FACTOR recovers the prime factors of a weak RSA modulus and uses them to derive the private exponent.

Analyst Interpretation:

In this example, the RSA modulus is intentionally small and vulnerable. RSA-FACTOR recovers the prime factors, calculates Euler's Totient Function, derives the private exponent, and attempts to recover the plaintext. This workflow demonstrates the relationship between factorization and RSA private-key recovery. Once p and q are known, the rest of the private key can be reconstructed.

Investigation Notes:

RSA-FACTOR is most useful when:

- The modulus is small
- The prime factors are close together
- The challenge is designed for Fermat-style recovery
- The RSA parameters are intentionally weak
- Educational visibility into RSA recovery steps is needed

RSA-FACTOR should be treated as an educational and challenge-solving tool rather than a general-purpose attack against properly generated modern RSA keys.

10.4 RSA-RHO

Purpose:

RSA-RHO implements Pollard Rho factorization. Pollard Rho is a probabilistic factorization method that can be effective against certain weak RSA moduli, especially when one factor is relatively small or the modulus has challenge-friendly structure.

Unlike Fermat factorization, Pollard Rho does not depend on the two prime factors being close together. This makes RSA-RHO useful as a second factorization attempt when RSA-FACTOR is unsuccessful.

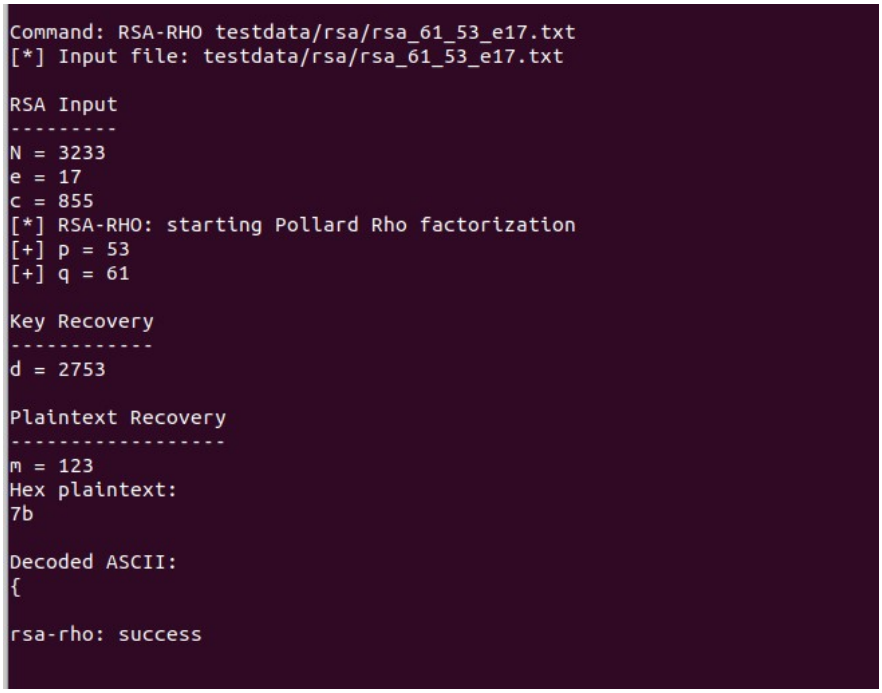
Usage:

```
./bin/k1wi RSA-RHO <rsa_file>
```

Example:

```
./bin/k1wi RSA-RHO testdata/rsa/rsa_61_53_e17.txt
```

Figure 10.2 – RSA-RHO Factor Recovery



RSA-RHO uses Pollard Rho factorization to recover a factor from a vulnerable RSA modulus.

Analyst Interpretation:

Pollard Rho searches for a non-trivial factor of the modulus using a cycle-detection approach. When a factor is found, K1Wi computes the corresponding cofactor and can continue the RSA recovery process. RSA-RHO is useful when classical factorization does not immediately succeed or when the challenge is designed around probabilistic factor recovery.

Investigation Notes:

RSA-RHO is most useful when:

- RSA-FACTOR does not recover factors quickly
- One factor may be smaller than expected
- The modulus is intentionally weak
- A probabilistic factorization approach is appropriate
- The challenge is designed for Pollard Rho recovery

Because Pollard Rho is probabilistic, runtime and success may vary depending on the modulus structure and implementation parameters.

10.5 RSA-ECM

Purpose:

RSA-ECM implements Elliptic Curve Method factorization. ECM is useful for finding relatively small factors of large composite numbers and can succeed in cases where classical factorization or Pollard Rho may be ineffective.

The RSA-ECM module is especially useful for educational RSA challenges, weak semiprime analysis, and cases where a modulus may contain a smaller hidden factor.

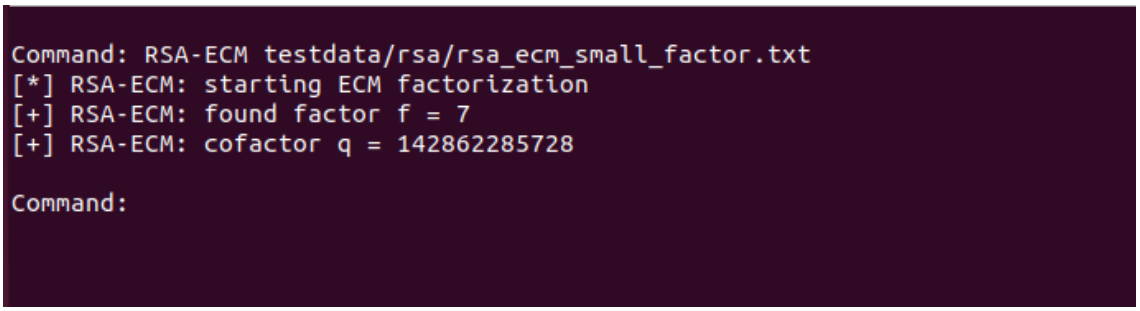
Usage:

```
./bin/k1wi RSA-ECM <rsa_file>
```

Examples:

```
./bin/k1wi RSA-ECM testdata/rsa/rsa_ecm_demo.txt
./bin/k1wi RSA-ECM testdata/rsa/rsa_ecm_demo.txt --curves 50
./bin/k1wi RSA-ECM testdata/rsa/rsa_ecm_demo.txt --bound 10000
./bin/k1wi RSA-ECM testdata/rsa/rsa_ecm_demo.txt --curves 50 --bound 10000
./bin/k1wi RSA-ECM testdata/rsa/rsa_ecm_demo.txt -c 50 -b 10000
./bin/k1wi RSA-ECM testdata/rsa/rsa_ecm_demo.txt --b1 10000
```

Figure 10.3 – RSA-ECM Factor Recovery



RSA-ECM uses Elliptic Curve Method factorization to recover a factor from a vulnerable RSA modulus.

Analyst Interpretation:

The Elliptic Curve Method attempts factorization by testing multiple elliptic curves and a Stage 1 bound. Increasing the number of curves or increasing the bound may improve the chance of finding a factor, but it may also increase runtime. RSA-ECM provides an additional factorization strategy when RSA-FACTOR and RSA-RHO do not succeed. It is especially useful when the modulus contains a factor that is small enough for ECM-style recovery.

Bound Controls:

RSA-ECM supports configurable curve and Stage 1 bound settings. These options make the ECM helper more useful for larger semiprimes or cases where the default search is insufficient.

Supported forms:

```
RSA-ECM <rsa_file>
RSA-ECM <rsa_file> --curves <N>
RSA-ECM <rsa_file> --bound <N>
RSA-ECM <rsa_file> --curves <N> --bound <N>
RSA-ECM <rsa_file> -c <N> -b <N>
RSA-ECM <rsa_file> --b1 <N>
```

Options:

--curves N

Controls the number of ECM curves attempted.

--bound N

Controls the Stage 1 bound B1.

--b1 N

Alias for --bound N.

-c N

Short alias for --curves N.

-b N

Short alias for --bound N.

The helper validates factor candidates and rejects invalid values, including 1, n, and non-divisors. This prevents false-positive factor reports.

Investigation Notes:

RSA-ECM is most useful when:

- RSA-FACTOR and RSA-RHO do not succeed
- The modulus may contain a smaller factor
- More configurable factorization attempts are needed
- A challenge is designed around ECM-style recovery
- The analyst wants to tune curve count or Stage 1 bound

RSA-ECM can be slower than simpler factorization methods, but it provides an important additional recovery path for weak RSA challenge material.

10.6 RSA-WIENER

Purpose:

RSA-WIENER attempts Wiener's continued fraction attack against RSA keys with an unusually small private exponent. This attack applies when the private exponent d is small enough relative to the modulus n.

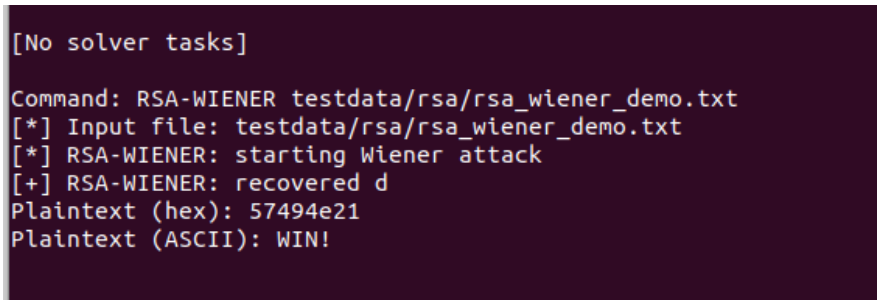
Usage:

```
./bin/k1wi RSA-WIENER <rsa_file>
```

Example:

```
./bin/k1wi RSA-WIENER testdata/rsa/rsa_wiener_demo.txt
```

Figure 10.4 – RSA-WIENER Private Exponent Recovery



RSA-WIENER attempts to recover the private exponent using Wiener's continued fraction method.

What it does:

RSA-WIENER analyzes the relationship between the public exponent e and modulus n. If the private exponent is vulnerable, the tool may recover d without directly factoring n first.

Best used when:

- The challenge hints at a small private exponent.
- RSA-FACTOR, RSA-RHO, or RSA-ECM are not the intended solution.
- The RSA key was intentionally generated with weak exponent parameters.
- The analyst wants to demonstrate Wiener's attack in an educational setting.

Wiener's attack does not apply to properly generated RSA keys with securely sized private exponents. It is most useful for CTF challenges, classroom demonstrations, and intentionally vulnerable examples.

10.7 RSA-SMALL-E

Purpose:

RSA-SMALL-E analyzes RSA challenges that use a small public exponent, commonly e = 3. Small public exponents are not automatically insecure, but they can become vulnerable when messages are poorly padded, reused, or small enough that modular reduction does not meaningfully occur.

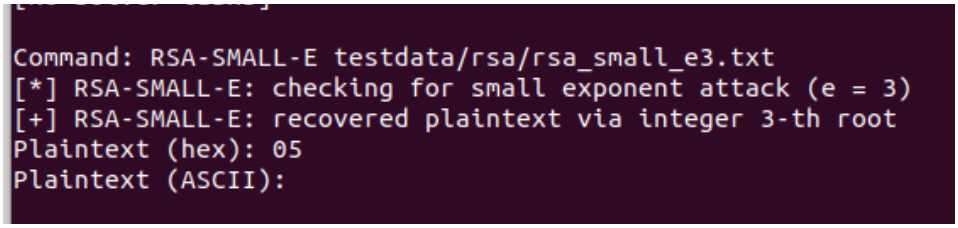
Usage:

```
./bin/k1wi RSA-SMALL-E <rsa_file>
```

Example:

```
./bin/k1wi RSA-SMALL-E testdata/rsa/rsa_small_e_demo.txt
```

Figure 10.5 – RSA-SMALL-E Recovery



RSA-SMALL-E attempts plaintext recovery from a weak low-exponent RSA challenge.

What it does:

RSA-SMALL-E checks whether a ciphertext can be recovered by applying low-exponent attack logic. In simple vulnerable cases, recovering the plaintext may require taking an integer root rather than factoring the modulus.

Best used when:

- The public exponent is small.
- The plaintext is small or poorly padded.
- The challenge suggests e = 3 or another low exponent weakness.
- The ciphertext may represent m^e without effective modular wraparound.
- The task is a CTF or educational RSA exercise.

Low-exponent attacks are normally prevented by correct padding schemes. RSA-SMALL-E is intended to demonstrate what can go wrong when RSA is used without proper padding or safe parameter choices.

10.8 RSA-KNOWNPQ

Purpose:

RSA-KNOWNPQ decrypts RSA challenge data when the prime factors p and q are already known. This is useful when factors have been recovered by another module, provided by a challenge, or discovered through manual analysis.

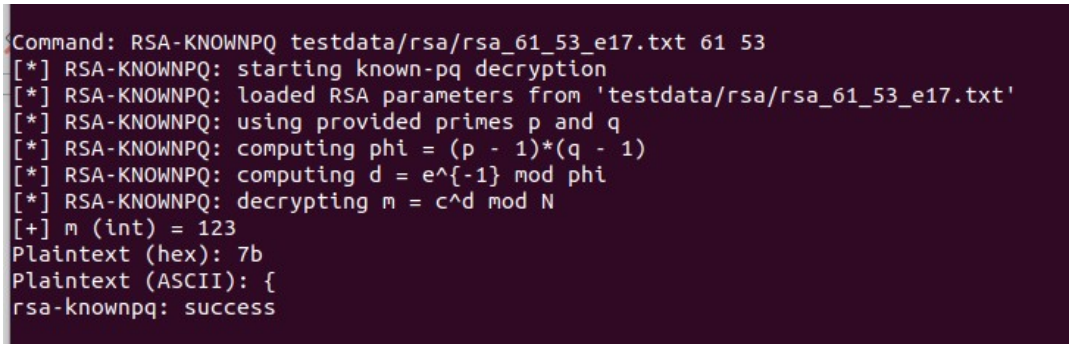
Usage:

```
./bin/k1wi RSA-KNOWNPQ <rsa_file>
```

Example:

```
./bin/k1wi RSA-KNOWNPQ testdata/rsa/rsa_knownpq_demo.txt
```

Figure 10.6 – RSA-KNOWNPQ Decryption



RSA-KNOWNPQ uses known prime factors to derive the private key and decrypt the cipher-text.

What it does:

RSA-KNOWNPQ reads known RSA parameters, computes φ(n), derives the private exponent d, and attempts to decrypt the ciphertext. This module demonstrates the full private-key reconstruction process once p and q are available.

Best used when:

- p and q are already known.
- Another K1Wi RSA module recovered the factors.
- A challenge provides the prime factors directly.
- The analyst wants to verify recovered RSA parameters.
- The goal is to demonstrate private-key derivation from known factors.

Once p and q are known, RSA no longer protects the message. RSA-KNOWNPQ makes that relationship clear by showing how known factors lead directly to private-key recovery.

10.9 RSA-CHECKPQ

Purpose:

RSA-CHECKPQ validates candidate RSA prime factors. It checks whether supplied values p and q correctly multiply to the modulus n.

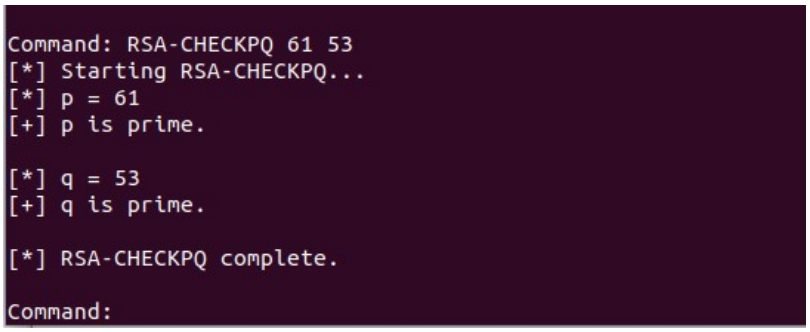
Usage:

```
./bin/k1wi RSA-CHECKPQ <rsa_file>
```

Example:

```
./bin/k1wi RSA-CHECKPQ testdata/rsa/rsa_checkpq_demo.txt
```

Figure 10.7 – RSA-CHECKPQ Factor Validation



RSA-CHECKPQ verifies whether candidate p and q values match the RSA modulus.

What it does:

RSA-CHECKPQ multiplies the supplied p and q values and compares the result against n. If the product matches, the factors are valid for the modulus. If they do not match, the candidate values are rejected.

Best used when:

- Candidate factors were recovered manually.
- Factors were copied from another tool or source.
- The analyst wants to verify p × q = n.
- A challenge provides possible factors that need validation.
- The user wants a quick sanity check before deriving d.

RSA-CHECKPQ is a simple but useful validation step. It helps prevent mistakes before moving on to private exponent recovery or decryption.

10.10 RSA-DFROMPQ

Purpose:

RSA-DFROMPQ computes the RSA private exponent d when p, q, and e are known. This module is useful after factor recovery or when a challenge provides the prime factors but not the private exponent.

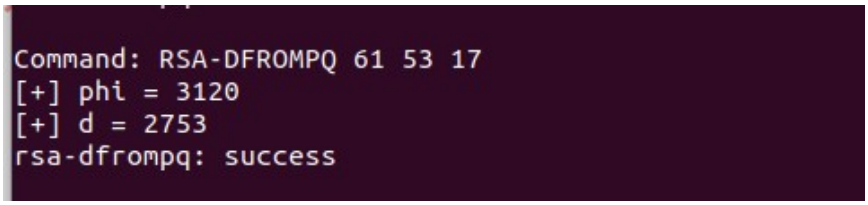
Usage:

```
./bin/k1wi RSA-DFROMPQ <rsa_file>
```

Example:

```
./bin/k1wi RSA-DFROMPQ testdata/rsa/rsa_dfrompq_demo.txt
```

Figure 10.8 – RSA-DFROMPQ Private Exponent Derivation



RSA-DFROMPQ derives the private exponent from known RSA prime factors and the public exponent.

What it does:

RSA-DFROMPQ computes Euler's Totient Function using p and q, then calculates d as the modular inverse of e modulo φ(n).

Formula:

$$\varphi(n) = (p - 1)(q - 1)$$

$$d \equiv e^{-1} \bmod \varphi(n)$$

Best used when:

- p and q are known.
- e is known.
- The private exponent d is missing.
- The analyst wants to reconstruct the private key.
- A challenge requires deriving d before decryption.

RSA-DFROMPQ focuses specifically on private exponent derivation. It is a useful bridge between factor validation and full RSA decryption.

10.11 RSA-MINI

Purpose:

RSA-MINI is an educational RSA helper designed for small, classroom-style RSA examples. It helps users understand RSA arithmetic without needing large parameters or complex setup.

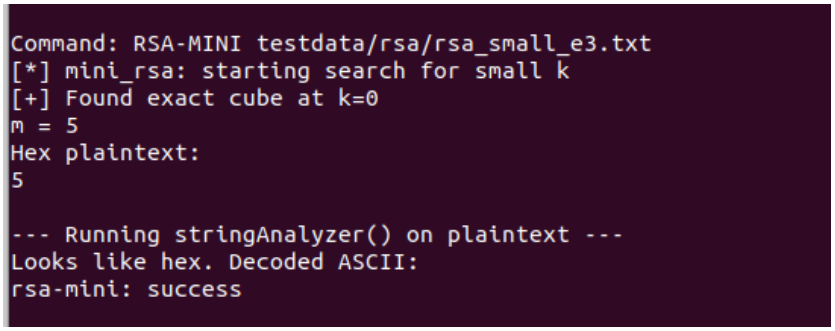
Usage:

```
./bin/k1wi RSA-MINI <rsa_file>
```

Example:

```
./bin/k1wi RSA-MINI testdata/rsa/rsa_mini_demo.txt
```

Figure 10.9 – RSA-MINI Educational RSA Walkthrough



RSA-MINI demonstrates RSA recovery steps using small educational parameters.

What it does:

RSA-MINI walks through simplified RSA calculations using small values. It may show factor recovery, totient calculation, private exponent derivation, ciphertext decryption, and recovered plaintext depending on the input file.

Best used when:

- Learning RSA fundamentals.
- Testing small RSA challenge files.
- Demonstrating how p, q, n, e, d, c, and m relate.
- Building confidence before using the larger RSA modules.
- Creating classroom, lab, or CTF training examples.

RSA-MINI is not intended for real-world cryptographic attacks. Its purpose is to make RSA mechanics easier to understand using small and transparent examples.

10.12 Practical RSA Workflow

A practical RSA analysis workflow depends on which values are known and what weakness is suspected. K1Wi provides multiple RSA modules so analysts can move through common challenge-solving paths without relying on a single technique.

A typical workflow may include the following steps:

1. Inspect the RSA challenge file and identify available values.
2. Determine whether n, e, c, p, q, or d are provided.
3. If p and q are known, validate them with RSA-CHECKPQ.
4. If p and q are known but d is missing, derive d with RSA-DFROMPQ.
5. If p and q are known and ciphertext is available, decrypt with RSA-KNOWNPQ.
6. If p and q are unknown, attempt factor recovery with RSA-FACTOR.
7. If RSA-FACTOR does not succeed, try RSA-RHO.
8. If additional factorization attempts are needed, try RSA-ECM with default or increased bounds.
9. If the challenge suggests a small private exponent, try RSA-WIENER.
10. If the challenge uses a small public exponent, try RSA-SMALL-E.
11. Use RSA-MINI for small educational examples or quick RSA arithmetic demonstrations.

Example workflow:

```
./bin/k1wi RSA-CHECKPQ testdata/rsa/rsa_checkpq_demo.txt
./bin/k1wi RSA-DFROMPQ testdata/rsa/rsa_dfrompq_demo.txt
./bin/k1wi RSA-KNOWNPQ testdata/rsa/rsa_knownpq_demo.txt
```

Factorization workflow:

```
./bin/k1wi RSA-FACTOR testdata/rsa/rsa_61_53_e17.txt
./bin/k1wi RSA-RHO testdata/rsa/rsa_61_53_e17.txt
./bin/k1wi RSA-ECM testdata/rsa/rsa_ecm_demo.txt --curves 50 --bound 10000
```

Weak-exponent workflow:

```
./bin/k1wi RSA-WIENER testdata/rsa/rsa_wiener_demo.txt
./bin/k1wi RSA-SMALL-E testdata/rsa/rsa_small_e_demo.txt
```

This workflow allows analysts to choose the correct RSA module based on the structure of the challenge rather than guessing blindly. Known-factor workflows should be attempted when p and q are available. Factorization workflows should be attempted when only n is known. Exponent-based workflows should be attempted when the challenge hints at weak exponent choices.

K1Wi's RSA modules are intended for educational analysis, CTF challenges, cryptographic learning, and authorized research. They should not be interpreted as general-purpose attacks against properly generated modern RSA keys.

10.13 Summary

The RSA Analysis Tool Suite gives K1Wi a broad set of educational cryptanalysis helpers for examining RSA challenge material. The suite includes factorization tools, weak-exponent tools, known-factor recovery tools, validation helpers, and small educational RSA workflows.

In this chapter, K1Wi demonstrated the following RSA capabilities:

- RSA-FACTOR for classical and Fermat-style factor recovery.
- RSA-RHO for Pollard Rho factorization.
- RSA-ECM for Elliptic Curve Method factorization with configurable curve and bound settings.
- RSA-WIENER for small-private-exponent recovery.
- RSA-SMALL-E for low-public-exponent challenge analysis.
- RSA-KNOWNPQ for decryption when p and q are known.
- RSA-CHECKPQ for validating candidate prime factors.
- RSA-DFROMPQ for deriving the private exponent from known p, q, and e.
- RSA-MINI for small educational RSA demonstrations.

Together, these modules provide a practical RSA workflow for students, researchers, and CTF participants. They help users understand how RSA parameters relate to each other, how weak configurations can fail, and how recovered factors can lead to private-key reconstruction and plaintext recovery.

The RSA tools are designed for transparency and learning. They are most useful in controlled environments where the RSA material is intentionally weak, educational, or part of an authorized investigation.

Chapter 11 – Utility Tools

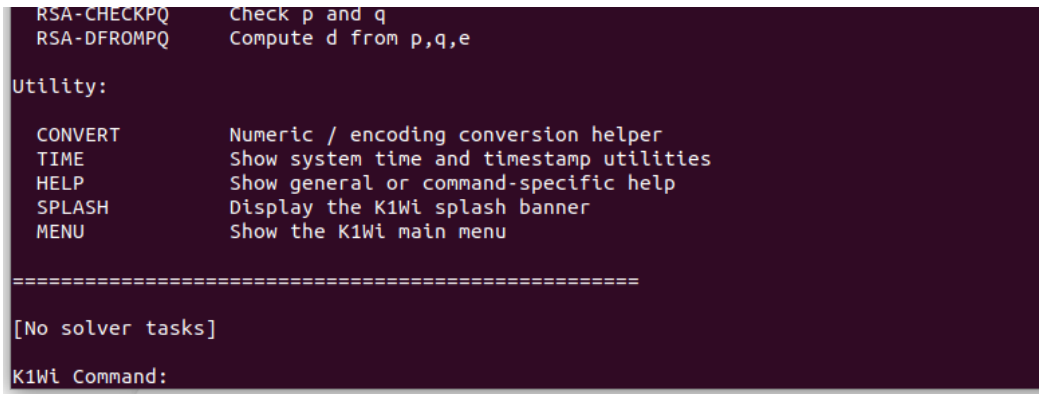
11.1 Overview

Utility tools provide small but important workflow helpers that support cryptography, forensics, reverse engineering, and Capture-The-Flag investigations. These tools are not full analysis engines by themselves, but they help users move data between formats required by other K1Wi modules.

In v1.2.0, the primary utility tool is CONVERT. CONVERT helps analysts translate between hexadecimal strings, integers, raw bytes, ASCII text, Base64, and SHA-256 digest integers.

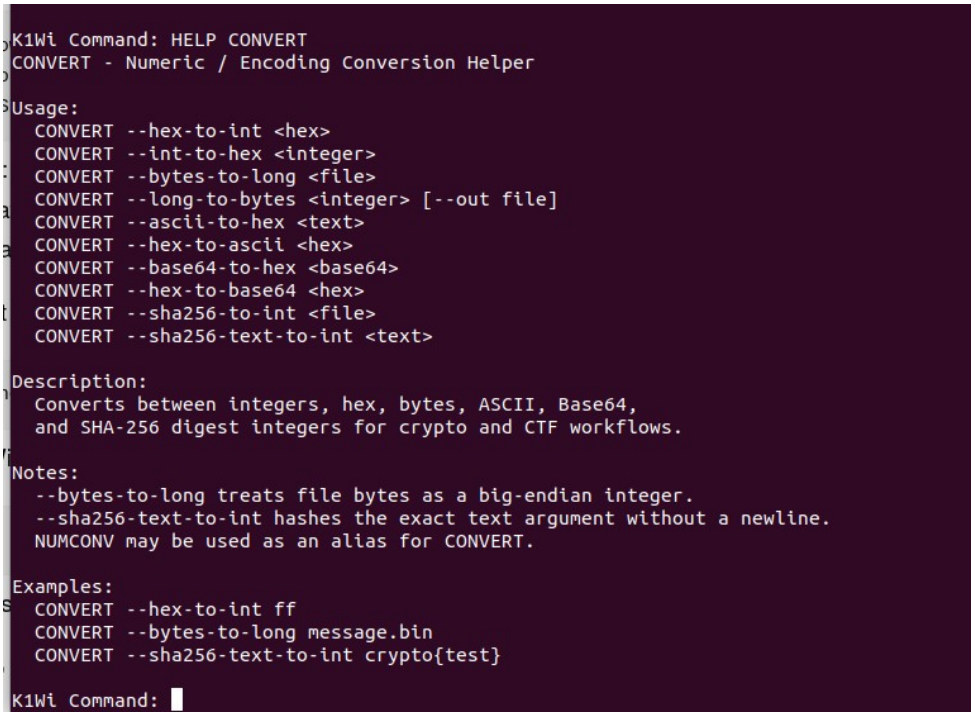
This is especially useful during RSA challenges, encoding analysis, file inspection, and workflows where data must be transformed before another module can process it.

Figure 11.1 – K1Wi Main Menu Showing CONVERT



The main K1Wi menu lists CONVERT as a utility command available from the command interface.

Figure 11.2 – CONVERT Help Output



The CONVERT help output displays supported conversion modes and command syntax.

11.2 CONVERT

CONVERT is K1Wi's numeric and encoding conversion helper. It is designed for workflows where analysts need to move between integers, hexadecimal strings, raw bytes, ASCII text, Base64, and SHA-256 digest integers.

Basic usage:

```
CONVERT --hex-to-int <hex>
CONVERT --int-to-hex <integer>
CONVERT --bytes-to-long <file>
CONVERT --long-to-bytes <integer> [--out file]
CONVERT --ascii-to-hex <text>
CONVERT --hex-to-ascii <hex>
CONVERT --base64-to-hex <base64>
CONVERT --hex-to-base64 <hex>
CONVERT --sha256-to-int <file>
CONVERT --sha256-text-to-int <text>
```

11.2.1 Hexadecimal and Integer Conversion

Hexadecimal and integer conversion is useful when translating between byte-oriented data and large integers used in cryptographic formulas.

Examples:

```
CONVERT --hex-to-int ff
CONVERT --int-to-hex 255
```

These commands convert between hexadecimal and decimal integer representations.

11.2.2 Byte Files and Large Integers

The bytes-to-long mode treats file bytes as a big-endian integer. This matches common cryptographic challenge behavior in Python-style bytes_to_long workflows.

Examples:

```
printf 'ABC' > /tmp/k1wi_convert_abc.bin
CONVERT --bytes-to-long /tmp/k1wi_convert_abc.bin
CONVERT --long-to-bytes 4276803
CONVERT --long-to-bytes 4276803 --out /tmp/k1wi_convert_out.bin
```

This workflow is useful when a challenge provides raw bytes that must be interpreted as an integer, or when a recovered integer must be converted back into bytes.

11.2.3 ASCII, Hex, and Base64

CONVERT supports common text and encoding transformations used during file analysis and CTF problem solving.

Examples:

```
CONVERT --ascii-to-hex K1Wi
CONVERT --hex-to-ascii 4b315769
CONVERT --base64-to-hex SzFXaQ==
CONVERT --hex-to-base64 4b315769
```

Hex inputs may contain spaces, colons, dashes, or underscores. For example, the following inputs all decode to ABC:

```
41 42 43
41:42:43
41-42-43
41_42_43
```

11.2.4 SHA-256 Digest Integers

The SHA-256 modes support RSA signing and verification workflows where a message digest must be interpreted as an integer.

Examples:

```
CONVERT --sha256-to-int message.txt
CONVERT --sha256-text-to-int crypto{Immut4ble_m3ssag1ng}
```

The file mode hashes the exact bytes contained in the file. The text mode hashes the exact text argument without adding a newline. This distinction is important because even a trailing newline changes the SHA-256 digest.

11.2.5 NUMCONV Alias

NUMCONV is accepted as an alias for CONVERT. It is intentionally hidden from the main menu to avoid clutter, but users may still type NUMCONV when they prefer the numeric-conversion name.

Example:

```
NUMCONV --hex-to-int ff
```

11.3 Practical CONVERT Workflow

A common RSA signing workflow requires converting a SHA-256 digest into an integer before applying RSA private-key arithmetic.

Typical workflow:

1. Hash the exact message bytes with SHA-256.
2. Convert the digest to an integer.
3. Compute the RSA signature using the private exponent.

Formula:

$$signature = digest^{d} \bmod n$$

CONVERT handles the digest-to-integer step directly and reduces the chance of mistakes caused by newline handling or manual conversion.

Example:

```
CONVERT --sha256-text-to-int crypto{Immut4ble_m3ssag1ng}
```

This command hashes the exact text argument and converts the resulting SHA-256 digest into an integer suitable for RSA-style signing or verification workflows.

11.4 MD5 and SHA256 Hash Utilities

K1Wi includes MD5 and SHA256 hashing utilities for file verification, integrity checking, duplicate comparison, and forensic triage. Hash values are commonly used during cybersecurity investigations to identify files, confirm that evidence has not changed, compare artifacts, and document analysis results. MD5 produces a shorter legacy hash value and is still commonly encountered in file identification workflows, malware databases, CTF challenges, and older forensic references. SHA256 produces a stronger modern cryptographic hash and is preferred for integrity verification and security-sensitive workflows.

Basic usage:

```
MD5 <file>
SHA256 <file>
```

Verify a file against a known hash:

```
MD5 -verify <file> <hash>
SHA256 -verify <file> <hash>
```

Compare two files:

```
MD5 -compare <file1> <file2>
SHA256 -compare <file1> <file2>
```

Example:

```
SHA256 testdata/archives/sample.zip
```

Hashing is useful when:

- Verifying downloaded or extracted files
- Comparing two files for equality
- Recording forensic evidence identifiers
- Checking whether a file changed during analysis
- Matching artifacts against known hash values
- Documenting CTF or investigation results

MD5 should not be used for modern security decisions because it is cryptographically broken for collision resistance. However, it remains useful for non-security file identification, legacy workflows, and compatibility with existing datasets. SHA256 should be preferred when integrity verification matters.

11.5 Summary

CONVERT provides K1Wi with practical utility conversions that support cryptography, encoding analysis, and CTF workflows. It helps users move between integers, hexadecimal values, raw bytes, ASCII text, Base64, and SHA-256 digest integers without leaving the K1Wi environment. The NUMCONV alias remains available for users who prefer a numeric-conversion command name, while CONVERT remains the primary documented utility command.

Chapter 12 – AUTO and MAGIC Updates

12.1 AUTO Input Detection

AUTO input detection improves K1Wi's ability to recognize common input patterns and route users toward the correct analysis workflow. Instead of requiring users to always know which module should process a value, AUTO-style detection helps identify whether input appears to be text, hexadecimal data, Base64, numeric data, binary-looking content, or another recognizable format.

This capability is useful during quick triage because analysts often begin with incomplete information. A challenge may provide only a short string, a suspicious blob of encoded data, or a value copied from another tool. AUTO detection helps reduce guesswork and supports faster movement into the appropriate analysis module.

AUTO detection may assist with:

- Identifying likely encodings
- Recognizing numeric or hexadecimal values
- Detecting binary-looking input
- Supporting CTF triage workflows
- Helping users choose the next command

AUTO detection should be treated as a helper rather than a final conclusion. Users should validate important findings with the appropriate K1Wi module, such as STRING, MAGIC, CONVERT, LYZER, or an RSA analysis command.

12.2 MAGIC Binary Digit Stream Detection

MAGIC identifies file types, signatures, and recognizable byte patterns. In v1.2.0, MAGIC includes improved handling for binary digit streams, allowing K1Wi to recognize inputs that appear to be sequences of binary digits.

Binary digit streams are common in CTF challenges, encoding exercises, and forensic puzzles. A user may encounter data represented as a long sequence of 0 and 1 characters rather than as raw bytes. MAGIC helps identify this pattern so the user can decide whether additional decoding or conversion is needed.

Example binary-looking input:

```
01001011 00110001 01010111 01101001
```

This sequence may represent ASCII text when interpreted as binary byte values. MAGIC detection does not automatically prove the meaning of the data, but it can alert the analyst that the input should be reviewed as a possible binary encoding.

MAGIC binary digit stream detection is useful when:

- A challenge provides long 0/1 strings
- Binary-looking data appears inside a file
- Encoded text may be represented as binary digits
- A user needs quick file or input triage
- MAGIC is used before deeper conversion or decoding

After MAGIC identifies binary-looking input, users may continue analysis with CONVERT, STRING, or manual decoding depending on the structure of the data.

12.3 Updated Regression Baseline

K1Wi v1.2.0 includes an updated regression baseline to validate the new and improved behavior added during this release cycle. Regression testing helps confirm that existing modules continue to operate correctly after new features, bug fixes, and documentation updates are added.

The regression suite validates major framework behavior, including:

- STRING detection
- LYZER output modes
- EXTRACT workflows
- PIECALC behavior
- ELFINFO output
- RSA analysis tools
- RSA-ECM bounds and aliases
- CONVERT and NUMCONV workflows
- AUTO and MAGIC detection behavior

A successful regression run should complete with zero failures.

Example:

```
./tests/run_regression.sh
```

Expected result:

```
PASS: 223
FAIL: 0
SKIP: 0
```

Regression testing is especially important before release because K1Wi combines many independent analysis modules. A clean regression baseline provides confidence that the framework remains stable after changes to command parsing, output formatting, detection logic, and analysis routines.

12.4 Summary

AUTO and MAGIC updates improve K1Wi's quick-triage capabilities by helping users recognize input types, binary-looking data, and common encoded patterns. These helpers support faster decision-making during CTF challenges, forensic review, and general analysis workflows. The updated regression baseline confirms that K1Wi v1.2.0 behavior is tested across the major tool areas and provides a release-quality validation checkpoint before packaging or publishing the manual.

Appendix A – Glossary

The glossary provides authoritative definitions for major terms used throughout the K1Wi User Manual. It serves as a quick-reference resource for students, researchers, analysts, developers, and investigators who require precise definitions of forensic, cryptographic, reverse engineering, and binary analysis terminology.

The definitions contained in this appendix are intended to provide concise technical explanations rather than exhaustive academic treatments. Where applicable, terms are described in the context of their use within the K1Wi Framework.

A–C

ASCII

A 7-bit character encoding standard used for representing text. In K1Wi, ASCII detection is part of the STRING Intelligence Engine.

ASLR (Address Space Layout Randomization)

A security mechanism that randomizes memory layout to prevent predictable exploitation. Relevant to **PIECALC**.

Base64

A binary-to-text encoding scheme commonly used to represent binary data as printable characters. K1Wi detects Base64 candidates during STRING analysis.

Block-Level Entropy

Entropy computed over fixed-size blocks to reveal localized randomness anomalies.

Carving

The process of extracting embedded files based on signatures and structural heuristics. Implemented in the **File Carver** subsystem.

Ciphertext

Data that has been encrypted and is unreadable without the appropriate decryption process or cryptographic key.

Chi-Square Test

A statistical test used in JPEG steganalysis to detect deviations from expected pixel distributions.

CTF (Capture-The-Flag)

A cybersecurity competition that presents participants with technical challenges involving cryptography, reverse engineering, digital forensics, exploitation, and software analysis.

D–F

DCT (Discrete Cosine Transform)

The mathematical transform used in JPEG compression. DCT coefficients are analyzed by K1Wi during JPEG forensic and steganalysis workflows.

Designation Block

A structured metadata container used within K1Wi. Designation blocks can be inspected and validated using the DESIG family of commands.

Difference-of-Squares

A factorization technique used in RSA analysis when primes are close.

Dynamic Library

A shared library loaded by an executable at runtime. Dynamic libraries provide reusable code and reduce executable size.

ELF (Executable and Linkable Format)

The standard executable file format used by Linux and many Unix-like operating systems. ELF files contain executable code, metadata, symbol information, libraries, and runtime loading information. ELF files are analyzed using the ELFINFO module.

ELF Header

The primary structure located at the beginning of an ELF file. It contains metadata describing file type, architecture, entry point address, and offsets to additional ELF structures.

Elliptic Curve Method (ECM)

A factorization algorithm that uses arithmetic performed on elliptic curves to discover non-trivial factors of composite integers.

Entropy

A measure of randomness. Used extensively in forensic analysis.

Extraction Engine

The recursive subsystem responsible for multi-layer unwrapping of files.

Fermat Factorization

A method for factoring numbers where primes are close together.

Frequency Analysis

A classical cryptanalytic technique used in substitution cipher solving.

G–L

GMP (GNU Multiple Precision Library)

A high-precision arithmetic library used by K1Wi for RSA analysis, factorization, and cryptographic calculations.

Heatmap (Entropy)

A visual representation of block-level entropy values.

Hillclimbing

An optimization technique used in cryptanalysis to refine keys.

Huffman Table

A component of JPEG compression used to efficiently encode image data. Huffman tables can be analyzed during JPEG forensic investigations.

IC (Index of Coincidence)

A statistical measure used to estimate Vigenère key length.

JPEG Stego Engine

The subsystem responsible for JPEG steganalysis.

K1Wi

An integrated cybersecurity analysis framework that combines digital forensics, reverse engineering, cryptanalysis, binary analysis, and Capture-The-Flag (CTF) tooling into a unified command-line environment.

Key Length Estimation

The process of determining likely Vigenère key lengths using IC and Kasiski.

Least Significant Bit (LSB)

The lowest-order bit in a binary value. LSB modification is commonly used in steganography and steganalysis investigations.

M–P

Magic Bytes

Signature bytes used to identify file formats.

Modulus (N)

The product of two prime numbers used in RSA cryptography. The modulus forms part of both the public and private keys.

PIE (Position-Independent Executable)

A binary that can be loaded at any memory address. Analyzed by **PIECALC**.

Plaintext

The decoded or decrypted form of a ciphertext.

Pollard Rho

A probabilistic integer factorization algorithm commonly used to recover factors from weak RSA challenge files.

Private Exponent (d)

The RSA private key component used during decryption. The private exponent is derived from the RSA prime factors and public exponent.

Probability Model (Quadgrams)

The statistical model used to score plaintext candidates.

Program Header

A structure within an ELF file that describes how executable segments should be loaded into memory during program execution.

Q–S

Quadgram

A sequence of four letters used in statistical scoring.

Recursion Guard

A safety mechanism that prevents infinite extraction loops.

Regular/Singular (RS) Analysis

A steganalysis technique that detects LSB embedding.

Return Address

A memory address used in binary execution and software exploitation. Return addresses are commonly examined during reverse engineering and PIE analysis workflows.

RSA

A public-key cryptosystem based on integer factorization.

Section Header

A structure that describes individual sections contained within an ELF file, such as code, data, symbols, and debugging information.

Shannon Entropy

The mathematical measure of randomness used throughout K1Wi for forensic analysis, entropy calculations, and steganalysis workflows.

String Intelligence

The subsystem responsible for extracting ASCII, UTF-8, base64, URLs, and emails.

Symbol Table

A collection of named functions, variables, and addresses contained within an executable or object file. Symbol tables are commonly used during reverse engineering and debugging.

T–Z

UTF-8

A variable-length character encoding standard. Detected by the String Intelligence Engine.

Vigenère Cipher

A polyalphabetic substitution cipher solved by **VIG**.

Wiener's Attack

A cryptanalytic attack that exploits RSA implementations using unusually small private exponents.

Zip Archive

A compressed archive format commonly encountered in multi-layer extraction workflows.

Appendix B – Command Quick Reference

This appendix provides a quick reference for commonly used K1Wi commands. It is intended as a compact lookup guide for users who already understand the purpose of each module and need a fast reminder of command syntax.

Core Commands

Command

```
./bin/k1wi –version
```

Purpose

Displays the current K1Wi version, build date, and build information.

Command

```
./bin/k1wi HELP
```

Purpose

Displays the built-in help system and available command categories.

Command

```
./bin/k1wi MENU
```

Purpose

Displays the interactive K1Wi command menu.

String Analysis Commands

Command

```
./bin/k1wi STRING "text"
```

Purpose

Analyzes a directly supplied string for printable text, encoding indicators, entropy, and suspicious artifacts.

Command

```
./bin/k1wi STRING –file
```

Purpose

Analyzes strings extracted from a file.

Command

```
./bin/k1wi STRING --decode
```

Purpose

Attempts to decode supported encoded content.

File and Forensic Analysis Commands

Command

```
./bin/k1wi ENTROPY <file>
```

Purpose

Calculates entropy for the supplied file.

Command

```
./bin/k1wi LYZER
```

Purpose

Performs image and file forensic analysis using the default LYZER workflow.

Command

```
./bin/k1wi LYZER <file> ALL
```

Purpose

Runs the full LYZER forensic analysis workflow, including entropy analysis, embedded signature detection, file carving, string intelligence, and steganography heuristics.

Command

```
./bin/k1wi EXTRACT
```

Purpose

Extracts supported content from an archive or container file.

Command

```
./bin/k1wi EXTRACT –recursive
```

Purpose

Performs recursive extraction against nested archives and discovered artifacts.

Binary and Reverse Engineering Commands

Command

```
./bin/k1wi ELFINFO -IN
```

Purpose

Displays ELF header information, program headers, dynamic libraries, and symbol tables for a Linux ELF binary.

Command

```
./bin/k1wi PIECALC --bin –list
```

Purpose

Lists available symbols and offsets from a PIE-enabled binary.

Command

```
./bin/k1wi PIECALC --bin –nearest
```

Purpose

Identifies the nearest known symbol to a supplied address.

Command

```
./bin/k1wi PIECALC --bin --json –list
```

Purpose

Generates symbol information in JSON format for scripting and automation.

RSA Analysis Commands

Command

```
./bin/k1wi RSA-FACTOR <rsa_file>
```

Purpose

Attempts to factor an RSA modulus and recover RSA private key parameters.

Command

```
./bin/k1wi RSA-RHO <rsa_file>
```

Purpose

Runs Pollard Rho factorization against a vulnerable RSA challenge file.

Command

```
./bin/k1wi RSA-ECM <rsa_file>
```

Purpose

Runs Elliptic Curve Method factorization against an RSA challenge file.

Command

```
./bin/k1wi RSA-WIENER <rsa_file>
```

Purpose

Attempts Wiener's continued fraction attack against RSA keys with unusually small private exponents.

Command

```
./bin/k1wi RSA-SMALL-E <rsa_file>
```

Purpose

Attempts plaintext recovery against RSA configurations using weak low public exponents.

Command

```
./bin/k1wi RSA-KNOWNPQ <rsa_file>
```

Purpose

Uses known RSA prime factors to reconstruct the private key and decrypt ciphertext.

Command

```
./bin/k1wi RSA-CHECKPQ
```

Purpose

Validates candidate RSA prime factors.

Command

```
./bin/k1wi RSA-DFROMPQ
```

Purpose

Computes Euler's Totient Function and derives the RSA private exponent from known p, q, and e values.

Command

```
./bin/k1wi RSA-MINI <rsa_file>
```

Purpose

Runs a simplified educational RSA helper for small challenge files.

Hashing and File Identification Commands

Command

```
./bin/k1wi MAGIC
```

Purpose

Identifies file type information using magic byte detection.

Command

```
./bin/k1wi MD5
```

Purpose

Computes the MD5 hash of a file.

Command

```
./bin/k1wi MD5 -verify
```

Purpose

Verifies a file against a supplied MD5 hash.

Command

```
./bin/k1wi MD5 -compare
```

Purpose

Compares two files using MD5 hashes.

Command

```
./bin/k1wi SHA256
```

Purpose

Computes the SHA-256 hash of a file.

Command

```
./bin/k1wi SHA256 -verify
```

Purpose

Verifies a file against a supplied SHA-256 hash.

Command

```
./bin/k1wi SHA256 -compare
```

Purpose

Compares two files using SHA-256 hashes.

Utility Conversion Commands

CONVERT Numeric, encoding, byte, Base64, and SHA-256 digest integer conversion helper.

NUMCONV Hidden alias for CONVERT. Accepted by the command parser but not displayed in the main menu.

AUTO and MAGIC Commands

AUTO Detects RSA, ECC, hashes, encodings, and encrypted challenge fields.

MAGIC Identifies file signatures and can recover decoded ASCII binary digit streams.

Testing and Build Commands

Command

```
make clean
```

Purpose

Removes previous build artifacts.

Command

```
make
```

Purpose

Compiles the K1Wi Framework.

Command

```
make debug
```

Purpose

Builds K1Wi with debugging symbols and development diagnostics.

Command

```
make release
```

Purpose

Builds K1Wi using release-oriented compiler settings.

Command

```
make sanitize
```

Purpose

Builds K1Wi with AddressSanitizer instrumentation.

Command

```
make tsan
```

Purpose

Builds K1Wi with ThreadSanitizer instrumentation.

Command

```
./tests/run_regression.sh
```

Purpose

Runs the K1Wi regression test suite.

Recommended First Commands

New users should begin with the following commands:

```
./bin/k1wi -version
./bin/k1wi HELP
./bin/k1wi STRING "hello world"
./bin/k1wi ENTROPY <file>
./bin/k1wi LYZER <file> ALL
./bin/k1wi ELFINFO -IN ./bin/k1wi
./bin/k1wi RSA-FACTOR testdata/rsa/rsa_61_53_e17.txt
```

Summary

This quick reference provides a compact command overview for common K1Wi workflows. For detailed explanations, examples, screenshots, and analyst interpretation, refer to the corresponding chapters in the main manual.